

Deep Learning

Filippo Merlo

2023-2024

Contents

Chapter 1	Introduction	Page 4
1.1	Supervised learning	4
	• Regression	
	• Classification	
1.2	Unsupervised learning	6
1.3	Reinforcement learning	6
Chapter 2	Neural Networks	Page 7
2.1	Simple example of Neural Network	7
2.2	Activation function	8
2.3	Loss function	10
	• Empirical Loss	
	• Binary Cross Entropy Loss	
	• Mean Squared Error Loss	
2.4	Training of Neural Networks	11
	• Loss Optimization	
	• Gradient Descent	
	• Random initialization	
Chapter 3	Neural Networks Improvements	Page 13
3.1	Hyperparameters	13
3.2	Train/Dev/Test sets	13
3.3	Underfit/Overfit	14
	• Regularization	
	• Dropout regularization	
	• Data augmentation	
	• Early stopping	
3.4	Normalization	17
	• Mean Normalization	
	• MinMax Normalization	
	• Comparison	
3.5	Vanishing/Exploding Gradients	18

Chapter 4 **CNN - Convolutional Neural Networks** _____ **Page 19** _____

4.1	Computer Vision	19
4.2	Convolutional filter	20
	• Padding	
	• Strided Convolution	
	• Multi-layered Convolution	
4.3	Convolutional Layers	23
	• Backpropagation with convolutions	
4.4	Pooling Layer	24
	• Max Pooling	
4.5	Transfer Learning	25
4.6	Modern Convolutional Networks Examples	25
	• AlexNet	
	• VGG16	
	• Residual Networks	
4.7	Localization and detection	28
	• YOLO - You Only Look Once	
	• IoU - Intersection over Union	
4.8	Image Segmentation	30

Chapter 5 **RNN - Recurrent Neural Networks** _____ **Page 31** _____

5.1	GRU - Gated Recurrent Unit	32
5.2	LSTM - Long Short-Term Memory	34

Chapter 6 **NLP - Natural Language Processing** _____ **Page 35** _____

6.1	Word Representation	35
6.2	Word2Vec	37
6.3	Transformers	38
	• Encoder	
	• Multi-Head Attention	
	• Training	
	• Fine-Tuning	
	• Decoder	
6.4	GPT	42
	• Masked Self-Attention	
	• ChatGPT	

Chapter 7 **GNN - Graph Neural Networks** _____ **Page 44** _____

7.1	Predictions on graphs	44
-----	-----------------------	----

7.2	GNN Models	45
7.3	Message Passing	45

Chapter 8 Optimization Page 47

8.1	Batch vs. Mini-Batch Gradient Descent	47
8.2	Algorithms	47
	• Exponentially weighted average • Gradient Descent with Momentum • RMSprop • ADAM - Adaptive Moment Estimation Optimizer	
8.3	Learning Rate	49
8.4	Local Optima in Neural Networks	50

Chapter 9 Loss functions Page 51

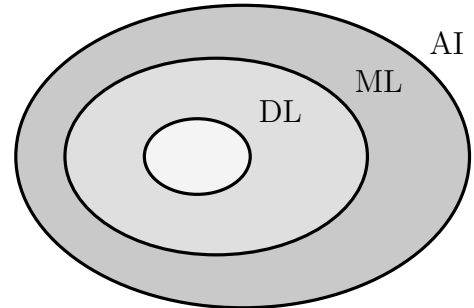
9.1	Softmax Activation Layer	51
9.2	Cross-Entropy Loss	51
9.3	Ranking Loss Function	51
9.4	Pairwise Loss Function	52

Chapter 1

Introduction

The general definitions for Artificial intelligence are:

- **Artificial Intelligence** = any technique which enables computers to mimic human behaviors.
- **Machine Learning** = uses statistical methods to improve the machine behavior.
- **Deep Learning** based on neural networks, extracts patterns from raw data to improve artificial intelligence.



There are different Machine Learning algorithms:

- Supervised learning
- Unsupervised learning
- Reinforcement learning
- Generative Learning

1.1 Supervised learning

A system that can solve specific tasks (one at a time). For an input there will be for sure an output (it doesn't matter if it's correct or wrong). The AI is trained and its behavior is checked by a real person. The files/images are labelled manually (e.g. 'face' or 'no face') by the staff before are given in input to the AI. All the files with their right classification constitute the dataset; in this way the AI can check its prediction against the right answer present in the dataset.

This type of learning can be furthermore split in two types:

- Regression
- Classification

1.1.1 Regression

As we can see in *Figure 1.1*, this AI makes numerical predictions.

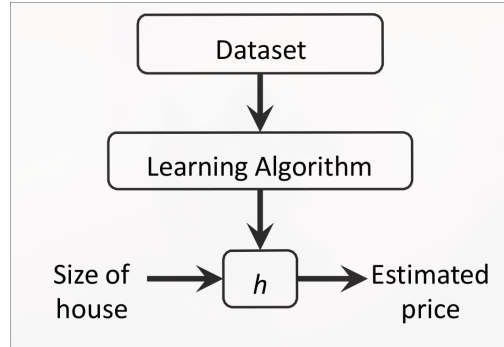


Figure 1.1: Regression mechanism
 h is the decision function

It can work over single or multiple features (parameters) of the dataset, like we can see in the two examples below (*Figure 1.2* and *1.3*). In the second case, it's easier for the AI to predict a precise response.

Size in feet ² (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

Figure 1.2: Single features

Size (feet ²)	Number of bedrooms	Number of floors	Age of home (years)	Price (\$1000)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

Figure 1.3: Multiple features

1.1.2 Classification

The classification problem is like the regression problem, except that the output domain is restricted and small. For example, in the **binary classification problem**, predictions can take on only two values: 0 (“negative class”) or 1 (“positive class”).

A real example of binary classification problem is the following one:

Suppose that we want to classify the tumor size in comparison to the patient age, a possible scenario is the one we can see in Figure 1.4 on the side.

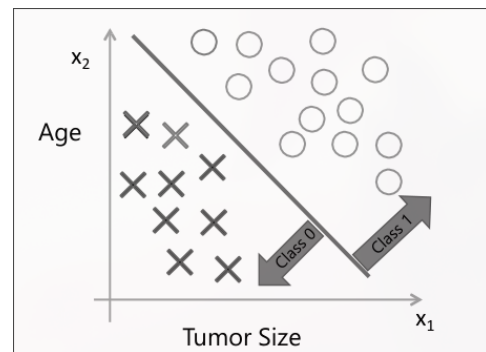


Figure 1.4: Tumor classification

The dataset for this type of problem is a set of tuples ($feature_1, feature_2$) and in the example above the tuples were ($age, size$).

We can train our AI to decide in which group a person will belong given its dataset. The AI is also responsible for deciding which is the most reliable and precise grouping (in our example above, the grouping function was the separating line).

1.2 Unsupervised learning

The main difference against the supervised type of learning, is that each result isn't labelled, thus we don't have correct or wrong predictions. The AI's task, in this type of learning, is to find a grouping of the input data. A **cluster** is a subset of the input data, which has been grouped because the AI found a similitude between them.

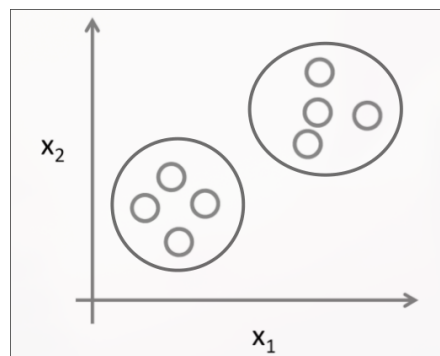


Figure 1.5: Market segmentation

The example in *Figure 1.5* above, is a market segmentation aimed to improve the selling quantities, based on the people buying.

Netflix, for instance, uses unsupervised learning with the movie thumbnails. It has 3 or 4 images for each film and with time, the AI is able to create as many clusters as the number of images provided and in each one there will be those people that liked more that thumbnail over the others. In this way, Netflix is able to decide which type of image is the most effective for you, based on your previous decision history, and show you only the type of thumbnail that you like the most, as a cover for its films.

1.3 Reinforcement learning

Like in robotics, to train machines to grab, transport or move objects (automation).

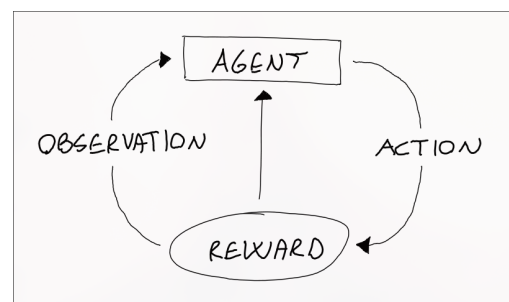


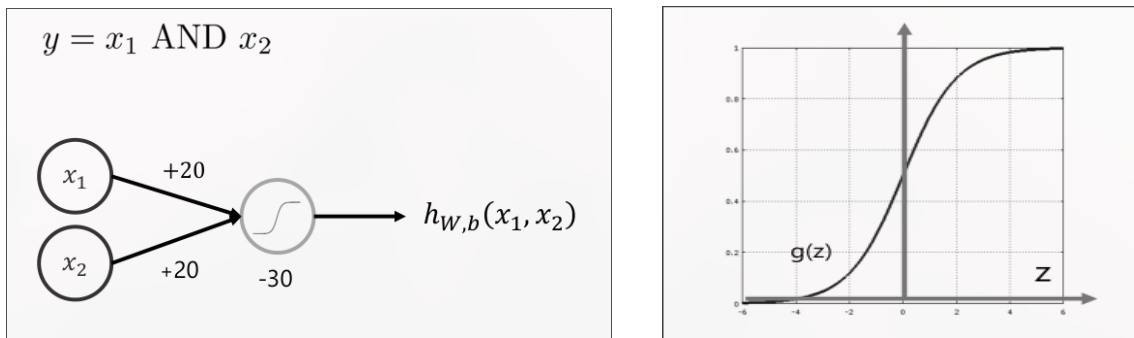
Figure 1.6: Reinforcement learning cycle

Chapter 2

Neural Networks

2.1 Simple example of Neural Network

A simple neural network example: AND logic port



The equation of the above neural network is

$$h_{w,b}(x_1, x_2) = g(wx_1 + wx_2 + b)$$

Where

- $\mathbf{x}$ is the input
- $\mathbf{y}$ is the output
- $\mathbf{h}$ is the activation function with $\mathbf{w}$ weight and $\mathbf{b}$ bias (error)
- $\mathbf{g}$ is the sigmoid function

Hidden layers add complexity without changing the model, weight or bias. With an additional intermediate layer, it's possible to add more weights and improve the Neural Network to solve more complex scenarios like in *figure 2.1*.

There will be cases not solvable also with a hidden layer. Adding neurons in the hidden layer is another way to increase furthermore the complexity, to make the AI able to solve and classify the data.

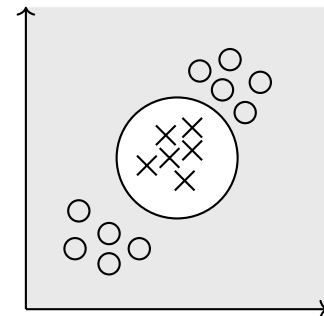


Figure 2.1: More complex scenario

The output layer has as many nodes as the number of classes.

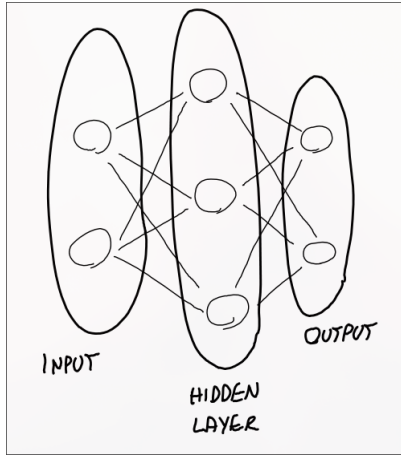


Figure 2.2: NN's layers

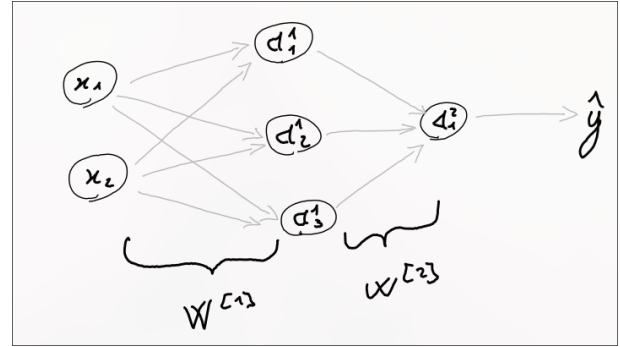


Figure 2.3: Matrixes of weights

As we can see in *Figure 2.3*

- $\hat{y}$ is the output
- $\mathbf{a}^{[j]}$ is the activation vector for layer j and its formula is $\mathbf{a}^{[j]} = g^{[j]} \cdot \mathbf{z}^{[j]}$, where $\mathbf{z}^{[j]}$ is the intermediate function value and its formula is $\mathbf{z}^{[j]} = \mathbf{W}^{[j]}x + \mathbf{b}^{[j]}$.
- $\mathbf{b}^{[j]}$ is the bias
- $g^{[j]}$ is the activation function for layer j , which puts the feature's values of $\mathbf{z}^{[j]}$ in the prediction space of $\mathbf{a}^{[j]}$. An example is the sigmoid function.
- $\mathbf{W}^{[j]}$ is the matrix of weights, mapping from layer $j - 1$ to layer j . In the figure $\mathbf{W}^{[1]}$ is a matrix 3×2 , while $\mathbf{W}^{[2]}$ has size 1×3 .

The training of a Neural Network that has been initialized with random weights, is performed by **backpropagation**. This technique consists in the repetition of the following steps:

1. **Network activation** (*forward step*): the network is activated and the error of the output layer is computed.
2. **Error propagation** (*backward step*): the error that has been computed is used to correct the weight values, starting from the output.

An **epoch** is a cycle of backpropagation, which corresponds to n back and forth steps (e.g. 10000), until all the dataset is used. Then, the cycle will be repeated another m number of times.

We are in the training phase, thus we use a dataset where each data is labelled, and we can establish if the prediction of the Neural Network was right or wrong.

2.2 Activation function

The most common activation functions are:

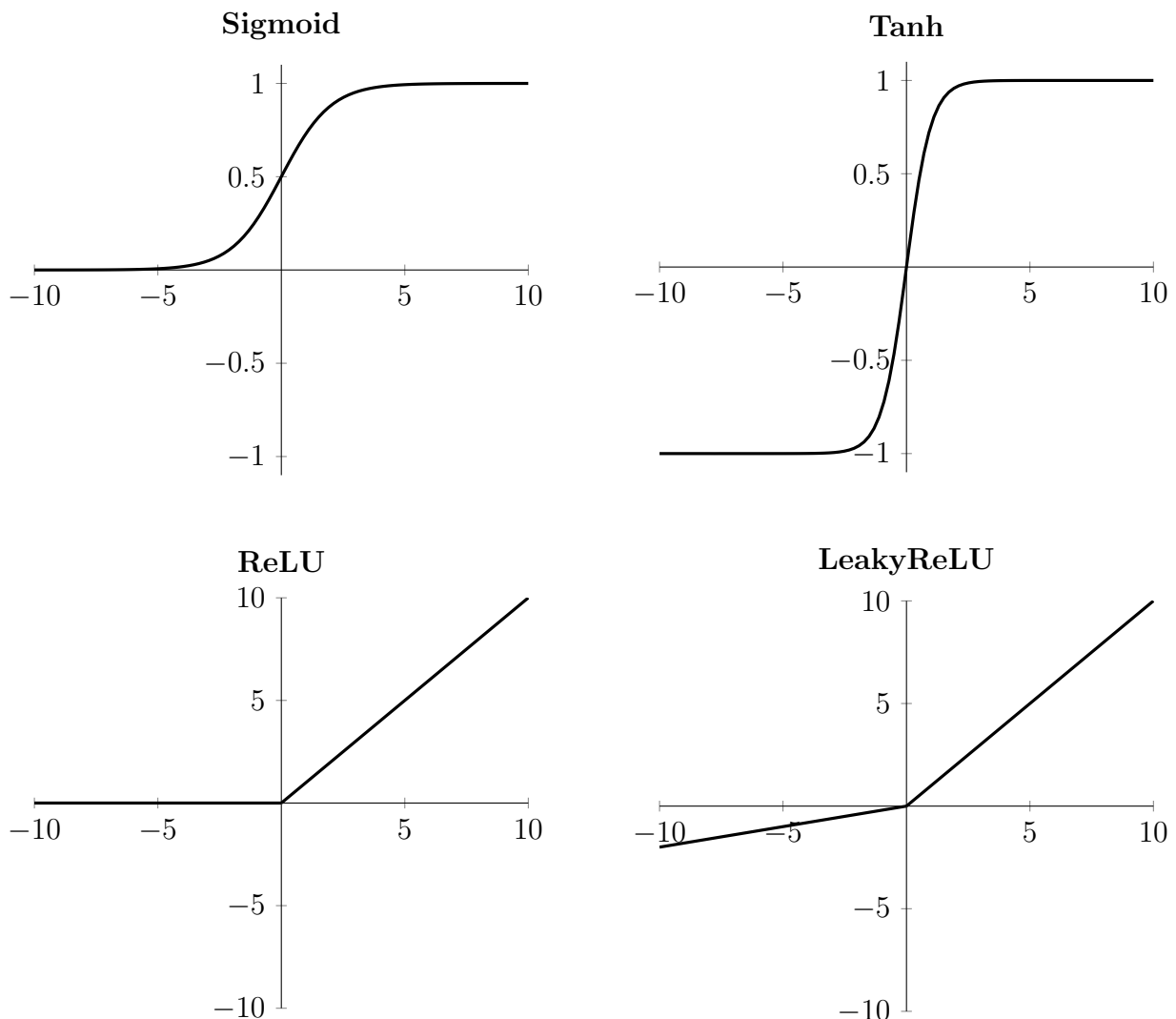
- **Sigmoid**: its formula is $f(x) = \frac{1}{1 + e^{-x}}$ which working space is $[0,1]$, thus is used mostly in the final layer before the output, to convert the prediction to that space.

- **Tanh:** $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ so its space is in $[-1,1]$.
- **ReLU** (*Rectified Linear Unit*): it's the most used and its definition is

$$y = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$

- **LeakyReLU:** is born to solve the **Dying ReLU problem**, where some ReLU “die” for all inputs, because the gradient of the function becomes 0 and the network cannot perform backpropagation and learn. For this reason a is set to 0.2 instead of 0.

$$y = \begin{cases} ax, & x \leq 0 \\ x, & x > 0 \end{cases}$$



Non-linear functions are needed because if before the 1st layer we don't have an activation function, in the next layer we will have a quadratic expression, not solvable through linear functions.

2.3 Loss function

It's also called **cost function** and it measures the “cost” of an incorrect prediction in a neural network. It's represented with $\mathcal{L}(\hat{y}, y)$, where $\hat{y}$ is the predicted value and y is the actual value associated to the given input.

2.3.1 Empirical Loss

We call **Empirical Loss** the total loss measured over the entire dataset and it's equal to

$$J(W, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

and this function is also called **objective function**.

2.3.2 Binary Cross Entropy Loss

Another useful function is the **Binary Cross Entropy Loss** and is a loss function used with models that output a probability between 0 and 1. Its formula is

$$J(W, b) = \frac{1}{m} \sum_{i=1}^m -[y^{(i)} \ln(\hat{y}^{(i)}) + (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)})]$$

The cost function is logarithmic.

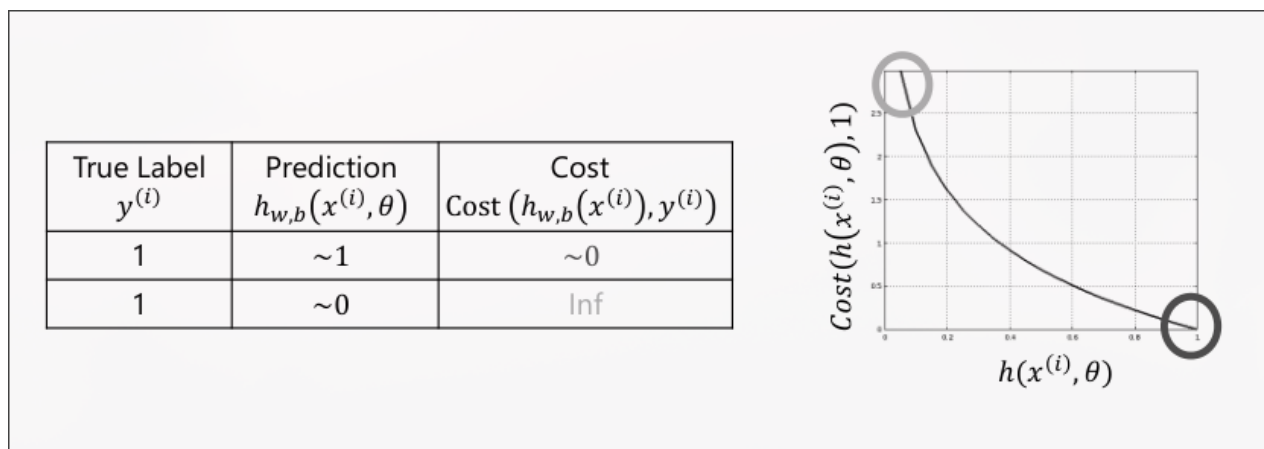


Figure 2.4: Cost function diagram

2.3.3 Mean Squared Error Loss

It can be used with regression models that output continuous real numbers and its formula is:

$$J(W, b) = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

2.4 Training of Neural Networks

2.4.1 Loss Optimization

It's the problem of finding the network weights and biases that achieve the lowest loss. The formula to compute it is

$$J(W, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$
$$[w^*, b^*] = \min_{(W, b)} J(W, b)$$

where the domain of the Neural Network are real numbers, $\mathcal{L}$ is the loss function, m is the number of samples and $[w^*, b^*]$ are the best weight and bias of the network.

This technique finds the **local minimum**, but we're not sure that the found value corresponds also to the global minimum.

2.4.2 Gradient Descent

The gradient descent is an algorithm to minimize a derivable function through linear regression. If the function is not derivable, the algorithm simply will not work. The algorithm is initialized with random weights and biases $(\theta_0, \dots, \theta_n)$, while, during the execution, these values are changed, in order to reduce the cost $J(\theta_0, \dots, \theta_n)$ and hopefully find a minimum. The algorithm code is:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

to be repeated until convergence. j are the indexes, α is the length/size of the step, J is the cost function and ∂ is the partial derivative.

NOTE:

The gradient of a function represents how small changes on the parameter θ_1 affect the cost function $J(\theta_1)$.

In the following schema we can see a computational graph of a neural network

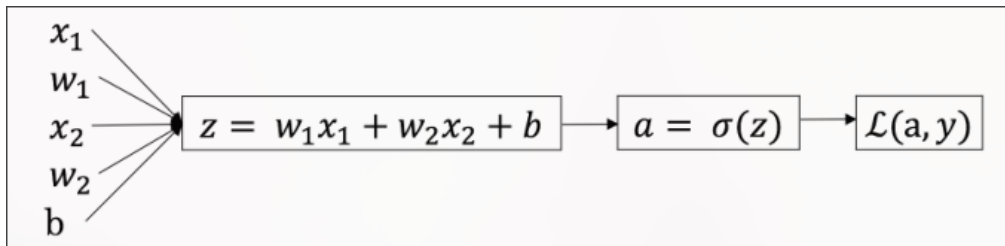


Figure 2.5: Computational graph

The first column are the inputs, the second column represents a neural network of type $z = w^T x + b$, in the third one is described the activation function a which is a sigmoid (σ) and the fourth and final column is the output, represented by a loss function.

To improve the loss function we need to change the 3 parameters w_1, w_2 and b and the new values of these parameters will be

$$\begin{aligned}
w_1 &:= w_1 - \alpha \frac{dJ(w, b)}{dw_1} \\
w_2 &:= w_2 - \alpha \frac{dJ(w, b)}{dw_2} \\
b &:= b - \alpha \frac{dJ(w, b)}{db}
\end{aligned}$$

where d is the derivative.

For instance, if we have the following function loss

$$Cost(a, y) = \mathcal{L}(a, y) = -[y \cdot \ln(a) + (1 - y) \cdot \ln(1 - a)]$$

the derivatives will be

$$\begin{aligned}
\frac{d\mathcal{L}}{da} &= -\frac{y}{a} + \frac{1-y}{1-a} \\
\frac{d\mathcal{L}}{dz} &= \frac{d\mathcal{L}}{da} \cdot \frac{da}{dz} = \left(-\frac{y}{a} + \frac{1-y}{1-a}\right) \cdot (a(1-a)) = -y + \cancel{ya} + a - \cancel{ya} = a - y \\
\frac{d\mathcal{L}}{dw_1} &= \frac{d\mathcal{L}}{dz} \cdot \frac{dz}{dw_1} = (a - y)x_1 \\
\frac{d\mathcal{L}}{dw_2} &= \frac{d\mathcal{L}}{dz} \cdot \frac{dz}{dw_2} = (a - y)x_2 \\
\frac{d\mathcal{L}}{db} &= \frac{d\mathcal{L}}{dz} \cdot \frac{dz}{db} = (a - y) \cdot 1 = (a - y)
\end{aligned}$$

Let us remember that the sigmoid function is $\sigma(x) = \frac{1}{1+e^{-x}}$, thus its derivative is $\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$.

2.4.3 Random initialization

If we initialize the weights to zero, it happens that all the neurons will have an equal value and this is a problem called **symmetry problem**. In fact, by initializing the weights to zero, each neuron in the first hidden layer will perform the same computation so, even after multiple iterations of gradient descent, each neuron in the layer will be computing the same thing as other neurons. A possible solution to this problem is to make a random initialization of the weights.

Biases, instead, don't suffer for the same problem and can be initialized to zero.

Chapter 3

Neural Networks Improvements

3.1 Hyperparameters

Hyperparameters are:

- learning rate
- number of epochs
- number of hidden layers
- number of hidden units (in each hidden layer)
- activation functions

All of these parameters are chosen empirically, following the cycle seen in *figure 3.1*

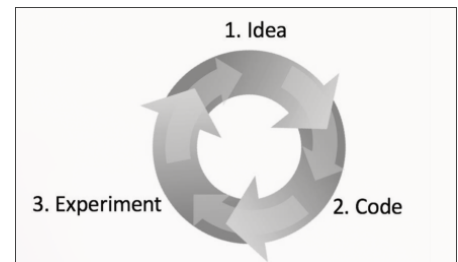
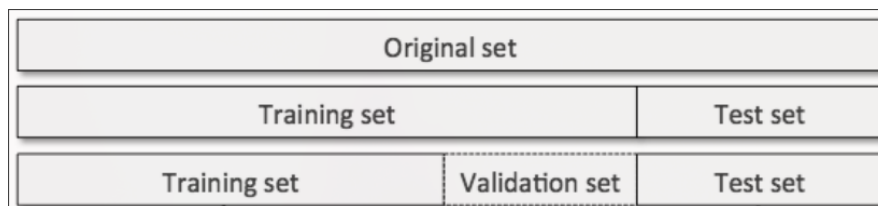


Figure 3.1: hyperparameters empirical cycle

3.2 Train/Dev/Test sets

The following image represents the distribution of the dataset over the different phases.



It's important that validation and test sets should come from the same distribution.

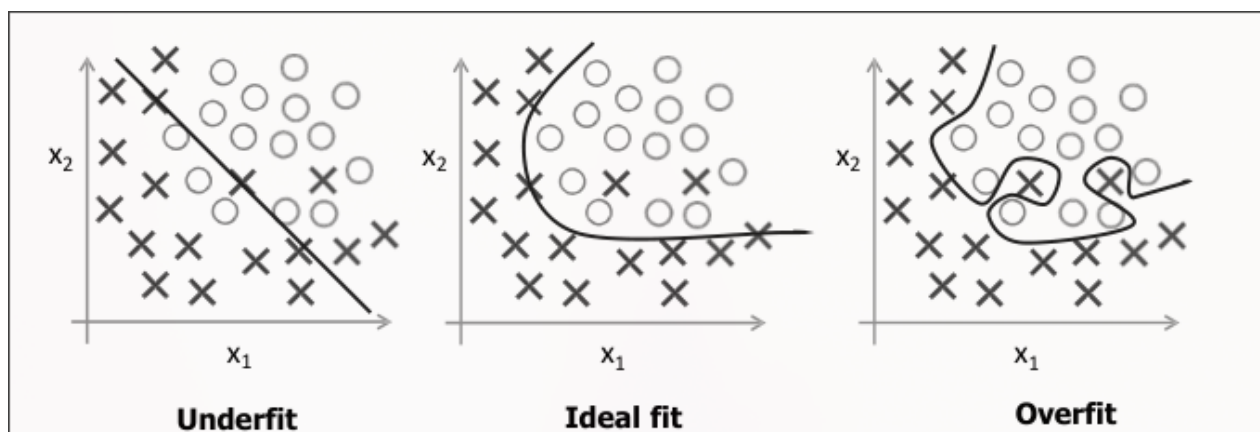
If the data set is small, we can enlarge it with other similar “general” data (for example taken from the web), but these data must be used only in the training set. For instance, if we want to recognize cat photos taken with a smartphone, we can use “general” cat photos in the training set, but in the validation and test sets we must use only images taken with a smartphone.

3.3 Underfit/Overfit

underfit = models don't have the capacity to fully learn the data.

overfit = too complex, doesn't generalize well.

Train set error:	1%	15%	0.5%
Dev set error:	11%	16%	1%
	Overfitting	Underfitting	Good*



As we can see from the two images above, overfitting is very precise, but even with a small change, it's performance decrease drastically. In the other hand, underfitting is more balanced and stable during the training and verification phases, but has higher error values than the ideal (good) case.

To solve underfitting we can try to:

- increase the network size, by making the network deeper or by increasing the number of units in each hidden layer
- prolong the training phase with additional epochs

Instead, to solve overfitting we can try to:

- use more training data (not so easy, it's a cost in terms of money and time)
- use regularization (increase the regularization parameter lambda; in this way the weights are pushed toward becoming smaller, so closer to 0)

Neural networks are more flexible for reducing underfitting/overfitting problems than the traditional Machine Learning approaches.

3.3.1 Regularization

The formula for the minimization problem with regularization is

$$\min_{(w,b)} J(w, b) \quad (3.1)$$

where the cost function $J(w, b)$ definition is

$$J(w, b) = \frac{1}{m} \left[\sum_{i=1}^m \mathcal{L}(\hat{y}^i, y^i) + \frac{\lambda}{2} \sum_{j=1}^n w_j^2 \right] \quad (3.2)$$

and λ is the **regularization parameter**. The bias b is ignored in regularization.

The first part of the cost function formula (3.2) is equal to the base one and its aim is to reach a final cost as close as possible to 0. The second part instead, tries to regularize the weights (have weights as close as possible to 0), otherwise we will have problem in the loss. If there are huge differences between the weights, there will be high chances that the activation function output will be based only on the big weights, while the small ones won't be considered. This translates into having a subset of the original neural network with only the big weights, losing precious information from the small weights. Regularization tries to avoid this scenario by using an average of the sum of the weights.

If we have multiple hidden layers then the second part of the formula will be instead

$$+ \frac{\lambda}{2} \sum_{l=1}^L \|W^{[l]}\|_F^2 \quad (3.3)$$

where l is the layer and

$$\|W^{[l]}\|_F^2 = \sum_{i=1}^{n^{[l-1]}} \sum_{j=1}^{n^{[l]}} (W_{ij}^{[l]})^2$$

The term 3.3 of the formula 3.2 with layers, is the sum of all the components that are in the weight's matrix.

The backpropagation with the regularization term is:

$$\frac{d\mathcal{L}}{dW^{[1]}} = \frac{d\mathcal{L}}{dz^{[1]}} \cdot x^T + \frac{\lambda}{m} \cdot W^{[1]}$$

where the second term is added because of the regularization and therefore is called **regularization effect**.

Regularization in neural networks is also called **weight decay**. In fact, it is a regularization technique (such as L2 regularization) that results in gradient descent shrinking the weights on every iteration.

$$\theta = \theta - \alpha \frac{dJ}{d\theta}$$

3.3.2 Dropout regularization

It's a common overfitting solution and this technique consists in **randomly ignoring** (or “dropping out”) some neurons in the hidden layers in order to **train** the model with a **smaller** network. This behavior makes the training process **noisier**, forcing nodes within a layer to probabilistically take responsibility of the input. This technique is effective because it randomly forces the nodes, to avoid big nodes from abusing the model.

The dropout can only be used in the training because in the other phases it's needed the whole network.

Usually are used higher probabilistic values (stronger dropout) in the first layers and lower values in the last ones, *i.e.* the dropout value is lowered as much as close we are to the output layer of the network.

3.3.3 Data augmentation

Data augmentation, in data analysis, are techniques used to increase the amount of data by adding slightly modified copies of already existing data or newly created synthetic data from existing data. The typical transformations are

- geometric transformations
- flipping
- color modification
- cropping
- rotation
- noise injection
- random erasing

3.3.4 Early stopping

When having a large network, there will be a point during training when the model will stop generalizing and start learning the statistical noise in the training dataset.

This overfitting of the training dataset will result in an increase in generalization error, making the model less useful at making predictions on new data.



One approach to solve this problem is to treat the number of training epochs as a hyperparameter and train the model multiple times with different values, then select the number of epochs that returned the best performance.

3.4 Normalization

3.4.1 Mean Normalization

It's the preferred normalization for Neural Networks, and it transforms data to have a mean $\mu = 0$ and a standard deviation $\sigma = 1$. Given a feature x_i the mapped feature is

$$x'_i = \frac{x_i - \mu_i}{\sigma_i}$$

where μ_i and σ_i are both computed on the training set and are respectively

Definition Mean

$$\mu_i = \frac{1}{m} \sum_{i=1}^m x_i$$

Definition Standard deviation

$$\sigma_i = \sqrt{\frac{1}{m-1} \cdot \sum_{i=1}^m (x_i - \mu_i)^2}$$

where m is the number of samples of the training set.

3.4.2 MinMax Normalization

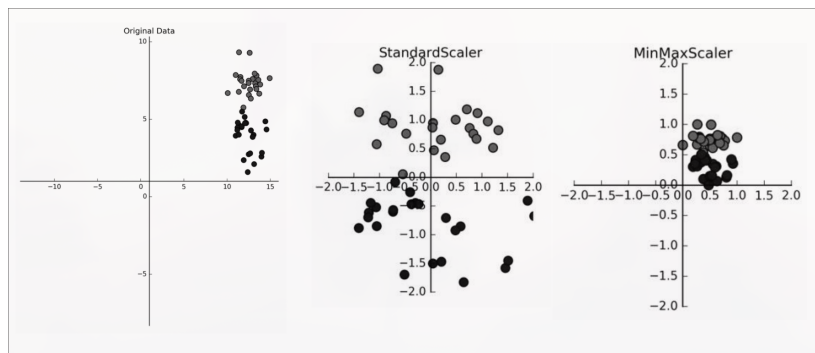
This normalization shifts the data such that all features are exactly between 0 and 1.

$$x'_i = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

x_i is the original value and x'_i is the normalized one.

3.4.3 Comparison

In the following image we can see a comparison between the two types of normalization, where the Mean is called StandardScaler and MinMax is the MinMaxScaler.



It's important to use the **same transformation** both for the training and the test, otherwise we could end up with two different scalings between the training and the test, like in the example in *Figure 3.3*.

Training Set			Training Set	
X	Y		X-Norm	Y
0	0	➡	0	0
5	10		0.5	10
10	20		1	20

Test Set			Test Set	
X	Y		X-Norm	Y
10	20	MinMax feature Normalization	1	20
15	30		1.5	30
20	40		2	40

Figure 3.2: Correct transformation

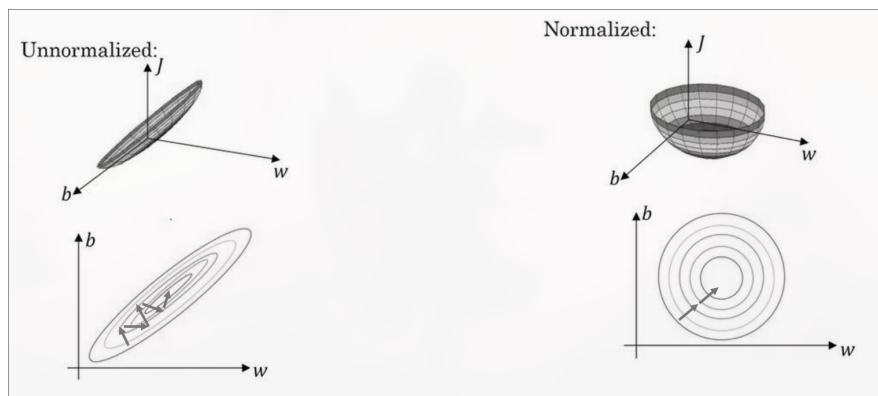
Training Set			Training Set	
X	Y		X-Norm	Y
0	0	➡	0	0
5	10		0.5	10
10	20		1	20

Test Set			Test Set	
X	Y		X-Norm	Y
10	20	MinMax feature Normalization	0	20
15	30		0.5	30
20	40		1	40

Figure 3.3: Incorrect transformation

As you can see, in *Figure 3.2*, has been used the **same values** (min = 0 and max = 10), while in *Figure 3.3* for the training has been used min = 0 and max = 10 and for the test min = 10 and max = 20. We can see immediately the difference, because the **same tuple** (10,20) of the dataset, has been scaled in two **different mappings** (1,20) and (0,20).

With normalized inputs the **gradient descent** will perform better, finding the minimum **efficiently**, as we can see in the image below, where the arrows represent the path followed by the gradient descent algorithm in order to find the minimum.



3.5 Vanishing/Exploding Gradients

Let's suppose we have a net with $g^{[i]}(z) = z$, $b^{[i]} = 0$ and L hidden layers, all equal to each other, for example $\begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}$, then the output will be $\hat{y} = \begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}^L$, which will have really large values. Instead, if the layers were equal to $\begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$, then the output will have small values. To resolve this problem, a solution is to set randomly the weights around 1.

Chapter 4

CNN - Convolutional Neural Networks

4.1 Computer Vision

In this field there are a lot of subfields, all related to the manipulation of data contained in images, for example:

- **image classification** - recognize if an image contains a specific object in order to classify it “*Does this image contain a cat? Yes/No*”
- **object detection** - find all the occurrences of an object in an image “*Find cars in the image*”
- **action recognition** - from a sequence of images recognize the ongoing action “*What is the person doing?*”
- **captioning** - brief textual description of the image “*A boy holding a toy hammer*”

SAM is a system able to segment images. This type of recognition is fundamental in the field of image generation/manipulation, because, with an image divided in segments, it is possible to replace only a specific segment of it (e.g. the segment of the subject or the segment of the background).

An image is a **tensor** (matrix) of integers in the range $[0, 255]$ where each number is a channel of the RGB¹ color-space of a specific pixel. For instance, with a colored image of 800×600 pixels, the quantity of integers needed to represent the photo in a computer is $800 \times 600 \times 3$. The main challenges for image detection are:

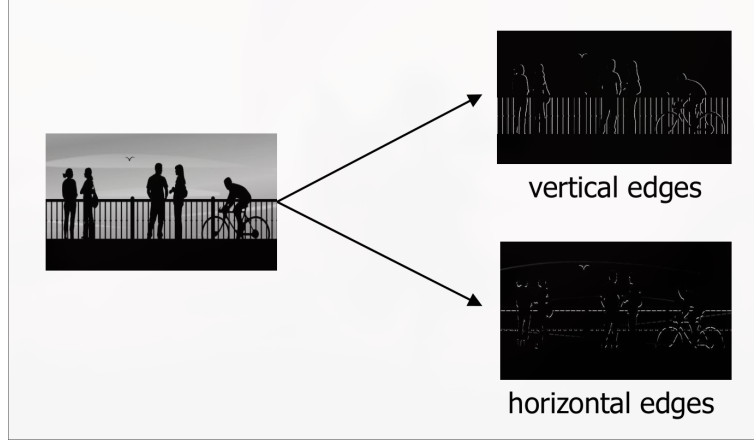
- **viewpoint variation** - even with a slight move in space of the camera, means that all the pixels change, so a totally different tensor, but a neural network model must be able to recognize the object anyway
- **background clutter** - this problem arises when the background has a chromatic gamma similar to the one of the subject (e.g. white cat in the snow)
- **illumination** - different light means that the camera records the colors differently, thus we have a complete different sets of pixels, even with the same exact position of the camera, object and environment, but just a different type of light (e.g. day vs night)
- **context** - also the context/environment around the subject and the position of it in the environment means different results
- **deformation** - for instance, a person can stay in different positions, and each one is a complete different image
- **intraclass variation**

¹Red Green Blue

4.2 Convolutional filter

Working on large images, for instance an HD picture that has 1920×1080 pixels ($1920 \times 1080 \times 3 > 6$ millions values), it becomes impossible using standard neural networks that we have seen until now, because, suppose that we have a small hidden layer of just 1000 weights, it means that there are over 6 billions edges between the input and the first hidden layer.

To solve this problem are born different solutions and for example, to recognize objects, a possible one is to work on the edges of the shapes present in the image. As we can see in the image below, we can extract only the data of the vertical (or horizontal) edges.



To do so, are used **convolutional filters**, which are matrixes with fixed values, depending on the result we want to obtain. Given the input data mapped into a matrix and a filter, the convolution is the output matrix of their inner product. The formula used to obtain the convolution is:

$$I(x, y) \cdot h = \sum_{i=-a}^a \sum_{j=-b}^b I(x-i, y-j) \cdot h(i, j)$$

where multiplication contained in the two nested sums is called **inner product**.

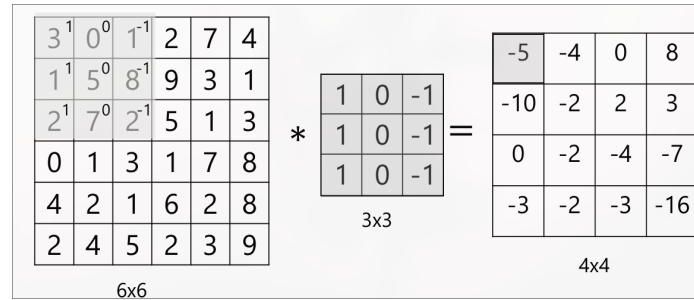


Figure 4.1: Example of convolution

The matrix on the left is composed of the input data (the data of the image), the small matrix in the center is the filter and the one on the right is the convolutional output.

The filter $\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$ is specialized to detect if there is a **vertical** edge in the input, while to detect **horizontal** edges is used the filter $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -1 & -1 & -1 \end{bmatrix}$.

In the image below there are other examples of filters for image manipulation

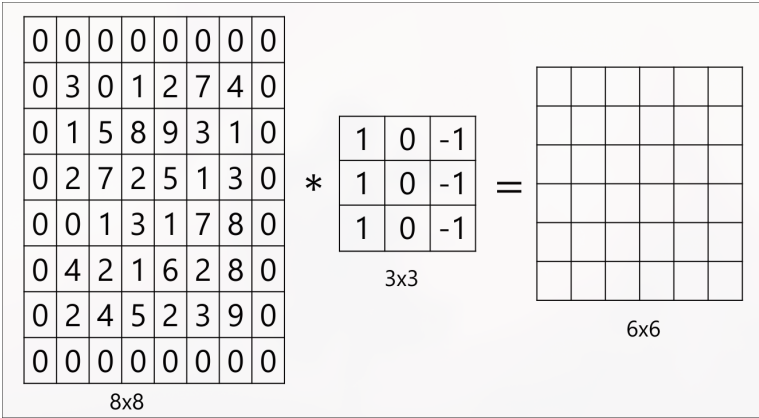


Figure 4.2: Examples of filters

Image processing and Computer Vision has many handcrafted filters like Canny, Sobel, Gaussian Blur, Morphological filters, Gabor Filters, etc...

4.2.1 Padding

Given a matrix input of size n and a filter of size m , the convolutional output matrix will be of size $n - m + 1$, which is equal to n only if the filter has size 1. In all the other cases the output matrix will be smaller and in most situations this is not a problem, but sometimes the model need a matrix of the same size. To solve this problem, has been invented the **padding** technique, which consists in adding $k \geq 0$ external layers of 0s around the original matrix, where $k = \frac{m-1}{2}$. In the example below, we can see that we had an input of 6×6 and a filter of 3×3 , but the output would have been a 4×4 matrix, so, to keep the original size in the output, we added $k = \frac{3-1}{2} = 1$ line of 0s all around the original matrix.



4.2.2 Strided Convolution

Sometimes when we have an image with a very big amount of pixels and those are also very similar to each other (have close values), while calculating the convolution, instead of moving of only 1 pixel at a time, we can set a **stride**, which corresponds to the number of pixels we need to move while calculating each cell of the convolution. Obviously the output matrix will have an even smaller size than the input, so it does pretty the opposite of the padding.

2	3	4	3	4	4	6	2	9
6	1	6	0	9	1	8	0	7
3	1	4	0	8	1	3	0	8
7	8	3	6	6	3	3	4	
4	2	1	8	3	4	4	6	
3	2	4	1	9	8	3		
0	1	3	9	2	1	4		

 $\ast$

3	4	4
1	0	2
-1	0	3

 $=$

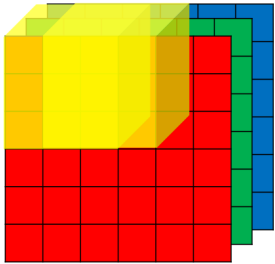
91	100	83
69	91	127
44	72	74

Figure 4.3: Example with stride=2

If we have an input matrix, where we have a stride that makes us go outside of it and not matching perfectly in the last columns, then we simply skip that convolution and proceed on a new line (if possible, otherwise we stop). For example, if in the above image we had set the stride to 3, then we will have covered columns from 1 to 3 in the first step, 4 to 6 in the second and in the last one we would have had only the 7th column, but we needed a total of three to go on, so we skip that and start a new line.

4.2.3 Multi-layered Convolution

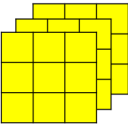
If we have multiple layers in the input matrix, the convolutional filter must have the same number of layers. For instance, if we work with colored images, these will be represented as a matrix of 3 layers (one for each RGB channel), so also the filter must be a 3-layered matrix.



6 x 6 x 3

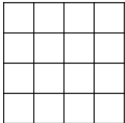
V. Edges on Red channel

 $\ast$



3 x 3 x 3

 $=$



4 x 4

R

1	0	-1
1	0	-1
1	0	-1

G

0	0	0
0	0	0
0	0	0

B

0	0	0
0	0	0
0	0	0

Figure 4.4: Example of RGB convolution

As we can see in the image, the output is a single layer matrix, so the image represented by that matrix will have only 1 channel.

4.3 Convolutional Layers

Are convolutions applied to entire neural networks. The network is equal to one with a traditional approach, but the number of edges is lower. A convolutional layer is a network with one or more filters, where the bias is applied to each convolutional output and then is also applied an activation function (often ReLU). In these networks, edges are less because not all nodes are connected together, but only those reachable within the filter size (and the stride).

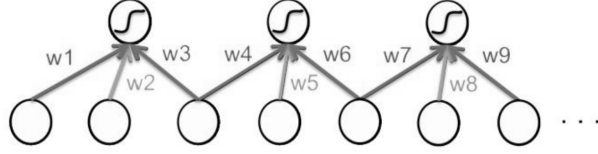


Figure 4.5: Example of a convolutional layer with a filter of size 3×3 and stride = 2

Convolutions reduce dimensionality, in fact, by applying convolutional layers, the images get smaller and smaller (e.g. in the image, after only two convolutional layers, the information is reduced to only 1 pixel) and this means losing details so the recognition accuracy drops.

Given two dataset S and T, where the first is fully annotated and plenty of images but not specific for our task, while the second is small but is precise for what we're looking for, we can build a model h_S and use it to make the model h_T learn better. This process is called **transfer learning**.

4.3.1 Backpropagation with convolutions

Let's suppose we are working on the same example of *Figure 4.5*, then the back propagation algorithm will work like this:

forward pass performs specific computations with all the weights $w_1, w_2, \dots, w_9$

backward pass first compute the gradients $\frac{\partial J}{\partial w_1}, \dots, \frac{\partial J}{\partial w_9}$ for all the weights and then update the weights using the average of the gradients from the shared weights

$$\begin{aligned} w_1 &= w_1 - \alpha \left(\frac{\partial J}{\partial w_1} + \frac{\partial J}{\partial w_4} + \frac{\partial J}{\partial w_7} \right) \\ w_4 &= w_4 - \alpha \left(\frac{\partial J}{\partial w_1} + \frac{\partial J}{\partial w_4} + \frac{\partial J}{\partial w_7} \right) \\ w_7 &= w_7 - \alpha \left(\frac{\partial J}{\partial w_1} + \frac{\partial J}{\partial w_4} + \frac{\partial J}{\partial w_7} \right) \end{aligned}$$

NOTE:

w_1, w_4 and w_7 because are on the same Red channel, so must be seen as a whole.

A convolutional layer must have set all the convolutional parameters (filter size, padding, stride and number of filters).

There exists multiple types of layer in Convolutional Networks, like:

- Convolution (just seen)
- Pooling
- Fully connected (other name to call standard NN)

4.4 Pooling Layer

4.4.1 Max Pooling

It's a simple operation to reduce the size of the input. Given some parameters (the same for convolution), we split the input using those values and then, for each section we just take the maximum value.

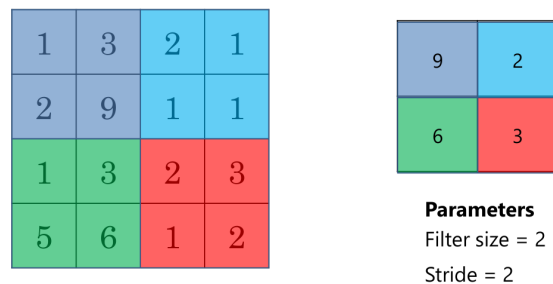


Figure 4.6: Example of Max Pooling

This layer, like said before, is used only to reduce the size (for example we only want to do a simple classification, and we don't need all the pixels, so we speed up the procedure by reducing a little bit the size of the image), because are used no weights or bias, thus the model doesn't learn.

Usually this layer is used after a convolution and that's the reason why we take only the max value. The idea of using this layer is to preserve the most important activation region (set of points) where the convolution filter found the most relevant points of the image.

With multiple layers, the pooling filter is applied on each channel independently.

Using a max pooling with a filter of size 2×2 and stride=2, is like doing the square root of the matrix input area, so dividing each size by 2, e.g. with an input matrix of size $32 \times 32 \times 28$, this max pooling layer will output a $16 \times 16 \times 28$ matrix.

4.5 Transfer Learning

Even if we have a dataset T which is not large, we can train a CNN for it without overfitting. We achieve this by pre-training a network over a dataset S, and then we can use:

- **fine-tuning**: start the training over T with the same weights and network obtained by the training over S. It's better to use this if S is large and T is medium, because if there are too few data it's better to fine-tune only the last layers (the fully connected layers), while if there are more data we can fine-tune also the convolutional layers.
- **CNN as feature extractors**: used when the dataset T is very small and the extracted features are used as input for a classifier. Taking a feature means removing the last output layer from the CNN network, giving in input the dataset T, and then we use the output vectors of the CNN as input for a classifier.

4.6 Modern Convolutional Networks Examples

In convolutional networks we usually see

- Multiple convolutional layers followed by a pooling layer.
- Fully-Connected layers in the last few layers.

4.6.1 AlexNet

Was a really popular Neural Network, until some years ago.

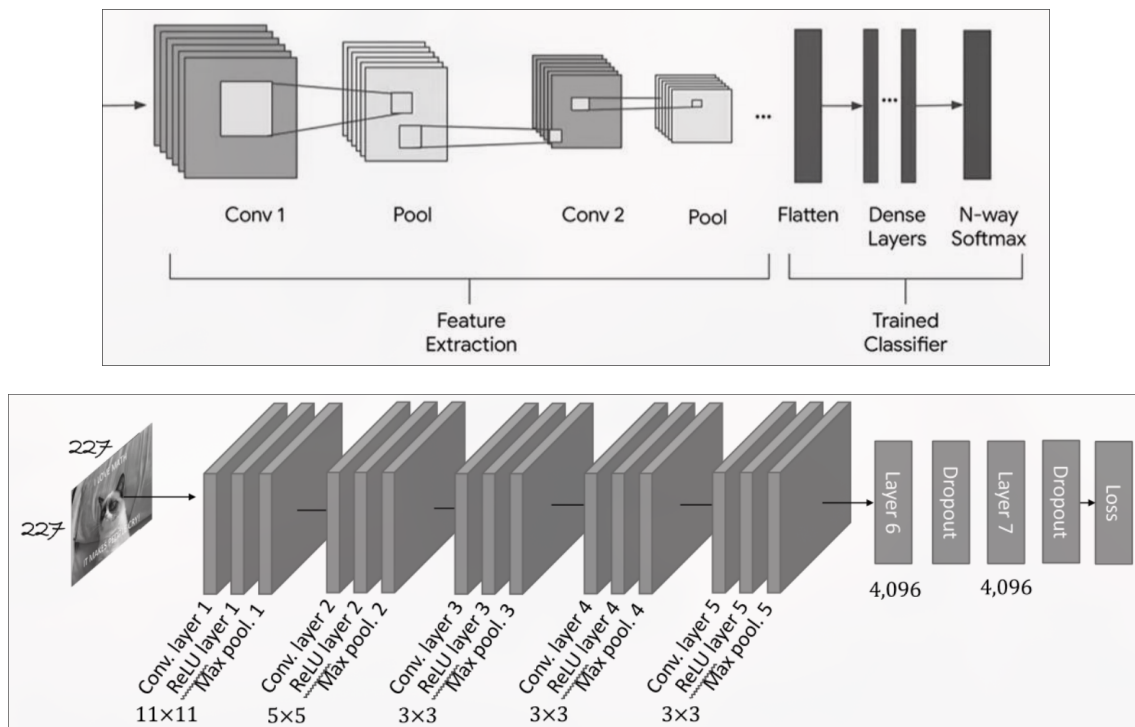
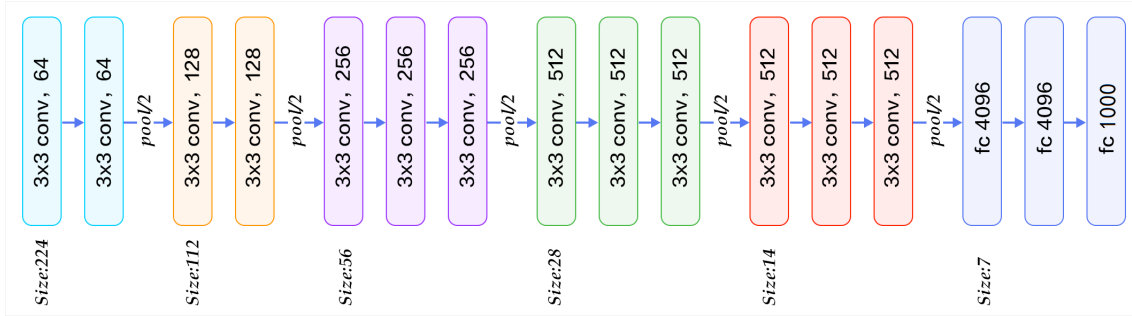


Figure 4.7: AlexNet architecture

The convolutional layers are the fundamental part of the network, removing one of these translates into a big drop in performance, with only a small reduction of parameters (3% drop, 1mln of parameters less). In the opposite, removing one or either the last two fully-connected layers, isn't a big problem in terms of performance (1.6/5% drop, but 16/50mln of fewer parameters).

4.6.2 VGG16

Is a network developed by the University of Oxford, which has achieved better results than AlexNet on image recognition (7.3% vs 18.2% of error rate).



In the network are used only 3×3 filters. Two 3×3 filters have the receptive field of one 5×5 filter, while three 3×3 of a 7×7 filter. However, using deeper stacks of smaller filters is more powerful than using a single larger filter and also generates fewer parameters (3 filters of 3×3 generates $3 \cdot 3 \cdot 3 = 27$ parameters, while a single 7×7 generates 49 parameters). Furthermore, because for each convolutional filter we have an activation function, having several convolutional filters means having a deeper network that adds non-linearity into the system.

4.6.3 Residual Networks

With residual networks, we're able to design very deep neural networks.

In the two images below, we can see the visual difference between blocks of plain network and a residual one:

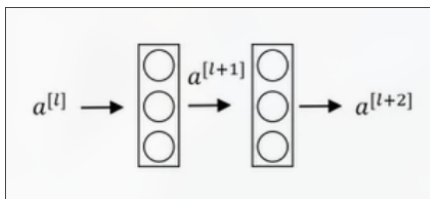


Figure 4.8: Plain

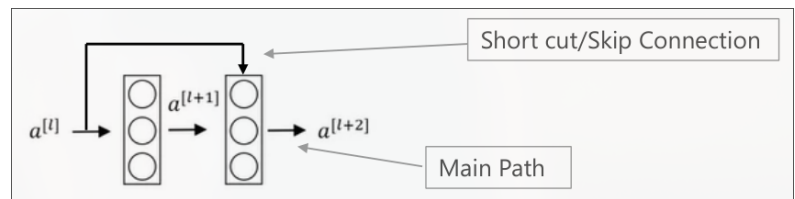
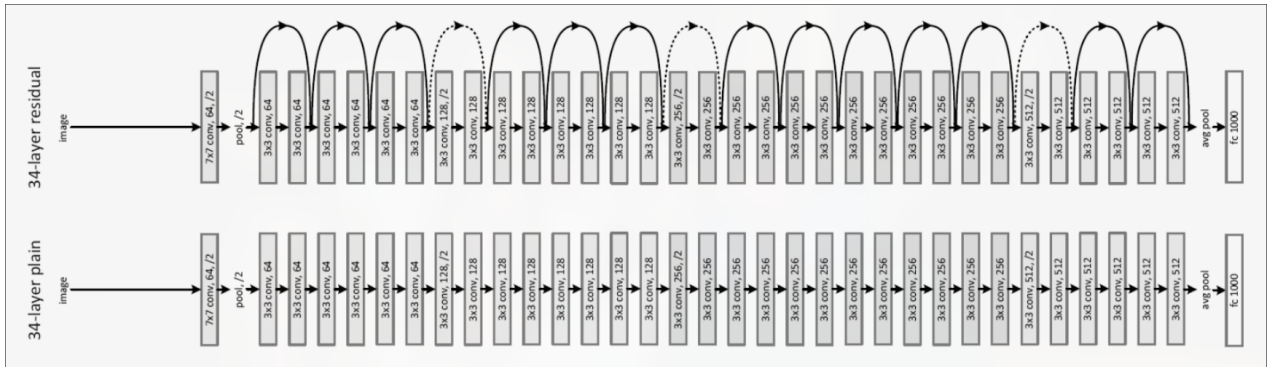
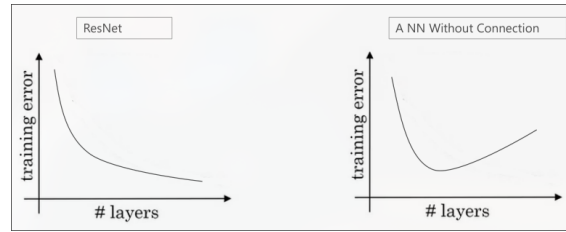


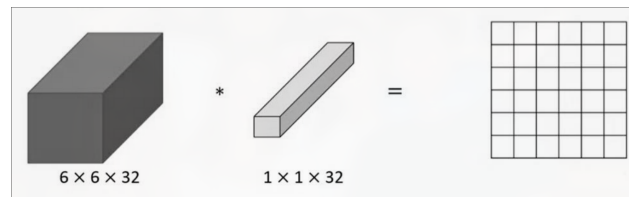
Figure 4.9: Residual

In the plain network we have that the value of $a^{[l+2]} = g(z^{[l+2]})$, while in the ResNet the formula slightly change into $a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$, where $z^{[l+2]} = W^{[l+2]} \cdot a^{[l+1]} + b^{[l+2]}$

In the images below we can first compare two graphs that represents how the error rate change with the number of layers, and in the second comparison how are represented two real networks.

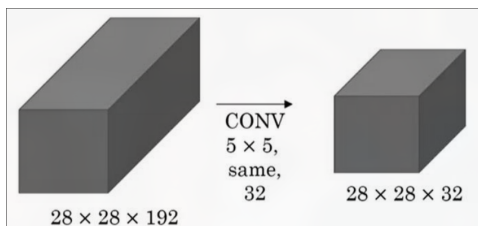


With Max pooling we're able to decrease the size of a single channel. If we want to reduce the number of channels we can use a 1×1 convolution.

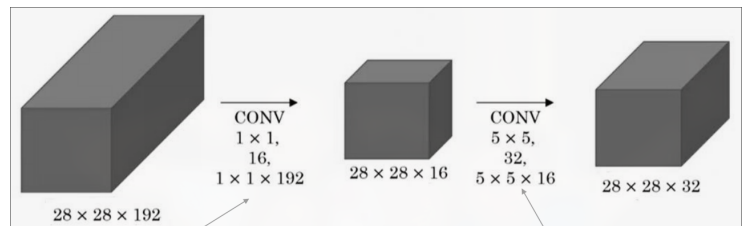


As we can see in the figure above, the output is a 2D matrix ($6 \times 6 \times 1$) and the channels has been reduced from 32 to 1.

If we wanted to generate an output that were $6 \times 6 \times 5$, we would have needed to use 5 convolutional layers of dimensionality $1 \times 1 \times 32$. These filters add also no-linearity factors. We can achieve the same result of using multiple 1×1 filters, by using a single larger filter, but this means that the computational cost increases, because we have more multiplications to do.



Single big filter



Two smaller filters

The computational cost of the filter in the left picture is $28 \times 28 \cdot 5 \times 5 \cdot 32 \cdot 192 = 120\text{M}$, while the cost of the two smaller filters on the right picture is respectively $28 \times 28 \cdot 1 \times 1 \cdot 16 \cdot 192 = 2.4\text{M}$ and $28 \times 28 \cdot 5 \times 5 \cdot 32 \cdot 16 = 10\text{M}$, so in total $2.4 + 10 = 12.4\text{M}$, which is circa 10 times less than using a single layer, but outputting the same result.

NOTE:

The “same” value specified in the filter on the left image, means that the filter is padded with enough zeros to reach the same dimensionality of the input.

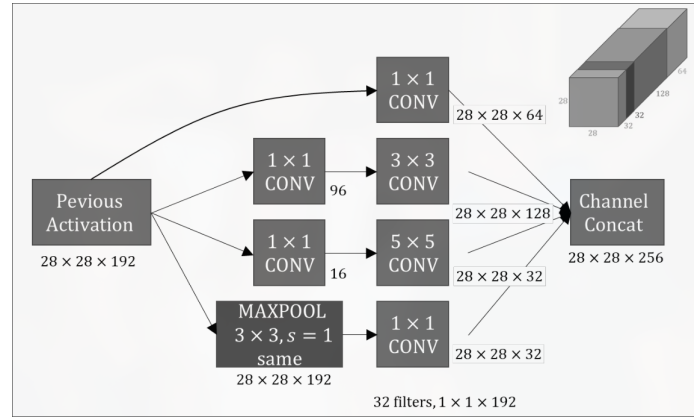
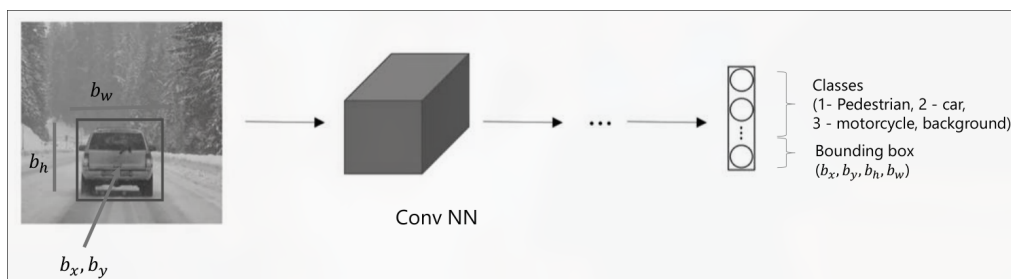


Figure 4.10: Inception module developed by Google

4.7 Localization and detection

Image classification, classifies images based on the object it detects in them. Exists also a variant **with localization** included, that after the detection, localizes the object by drawing a bounding box around it. **Detection** differs from the previous two because is image classification with localization but for multiple objects (the previous models where single object detection).

In an object localization network, the output is split in two, on one branch there is a **softmax** function for the classification, while on the other there is a **regression** function that finds the coordinates of the object’s bounding box. Like we can see in the picture below, the output is split between a part for the classification and another for the bbox.



The **loss**, if there is an object in the image, takes care of all the output values, otherwise it takes care only of the P_c value (the first one).

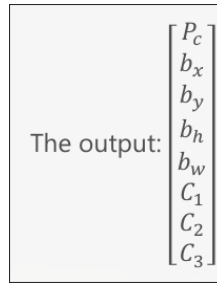


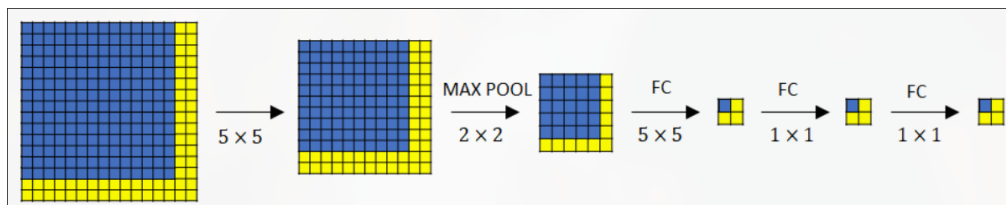
Figure 4.11: Example of output vector

Landmark detection's aim is to find specific points in an image, so is used, for example, in the recognition of people's faces or posture.

To be able to detect an object and also find its location, has been used the following idea: define a fixed size of the bounding box and use the box as a **sliding window** used to analyze only the portion of image isolated by the box and trying to detect any object inside. This approach, however, is not robust in cases where the object can have **different scales**. A possible patch to this problem is to scan multiple times the same image but with windows of different sizes or to **overlap** the boxes (usually is 50% of the width).

Using fully-connected layers, means that each pixel of the image influence every pixel in the following layer, while working with convolutional layers, if we select a specific portion of the image, then in the following layers, only a specific portion is strongly related to the previous portion and not the entire image.

Like we can see in the following figure, the top-left part of the image evidences the portions that are strongly related to each other, while proceeding in the network layers.



With fully-connected layers, we lose the spatial information.

Through the **spatial information** maintained by using convolutional layers, we're able to implement sliding windows, because we know that some specific points of the output depends on a specific (larger) portion of the input. Using this strong relation is key to be able to use the sliding window approach.

4.7.1 YOLO - You Only Look Once

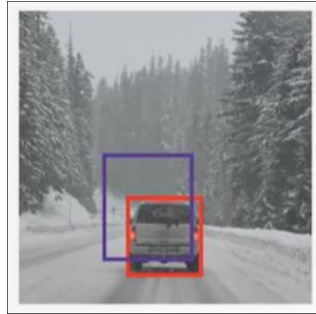
Using sliding windows led to another problem, which is that **not all** objects have a **square shape** (e.g. cars rectangular), so using square boxes give a not so accurate localization of them. To solve this problem has been developed the YOLO algorithm, which is based on the following idea: using a **grid** over the image, where for each grid cell is associated a **vector** and each detected object is associated to only one cell (the location of the centroid object determines the associated grid cell).

4.7.2 IoU - Intersection over Union

To estimate the precision of the area of the predicted object location, is used the **Intersection over Union** approach, which is

$$\text{Intersection over Union} = \frac{\text{area of intersection (overlap)}}{\text{area of union}}$$

This value returns the precision of the bbox respectively to the Ground-truth (the precise bbox annotation draw for the evaluation check).



In the figure above the blue (upper-left) box is the predicted bbox, while the other (the red down-right) is the Ground-truth one.

Furthermore, can happen that we have multiple detections of the same object, so multiple bounding boxes. To make the most precise deduction of the location is used the **Non-Max suppression** algorithm, which trivially discard all the detections that are not so secure. This is achieved by fixing a lower bound for the P_c value in the vector (e.g. $P_c > 0.6$, lower predictions are discarded). If there are still multiple boxes, we pick the box with the largest P_c output and then discard all the remaining boxes with $\text{IoU} > 0.5$ against the picked box. With multiple-object detection, this process must be run for each object independently.

4.8 Image Segmentation

Can be split in two sub-categories:

- **semantic** segmentation: find all the pixels that belong to a category (e.g. at a restaurant, detect the groups of people of the same table).
- **instance** segmentation: identify separately each single instance of a category (e.g. at a restaurant, detect each single person at the same table).

The architecture of image segmentation is based on two components (networks): an **encoder**, which extracts features by down-sampling the images (removes irrelevant details), and a **decoder** that takes in input the down-sampled images with only the important information and increases the dimension to reach back the original image size (up-samples the feature map).

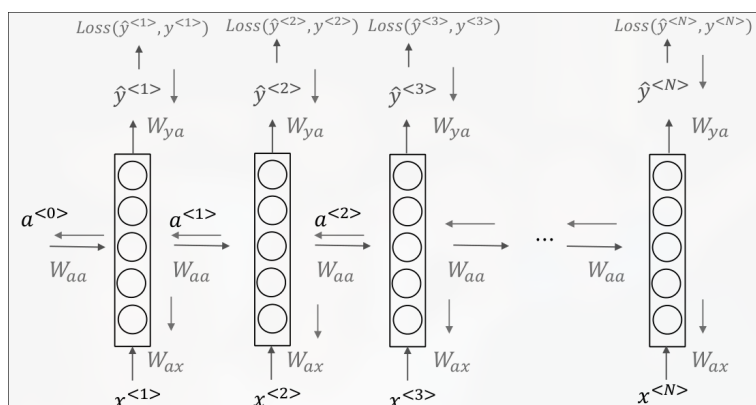
Among the scaling techniques there is the **up-sampling**, which scales up the image and is subdivided into two types:

- nearest: copies the value from the nearest pixel.
- bilinear: linear interpolation from nearby pixels.

Chapter 5

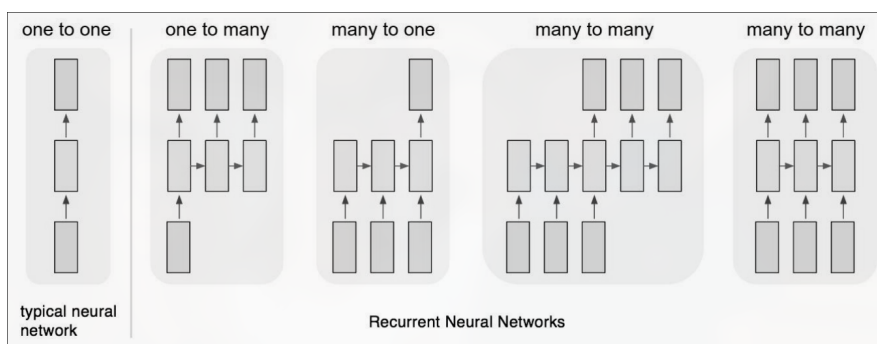
RNN - Recurrent Neural Networks

The image below is a schema of a recurrent neural network:



Each circle represents a **unit** (standard neural network with its own activation function), each rectangle is a hidden **layer**, x is the **input** of the layer and is fully-connected to each hidden state (the units in the layer), $\hat{y}$ is the layer's **output** prediction (also the output is fully-connected with each hidden state) and W are the **weights** that are shared. The weights sharing properties makes our network suitable for variable-size inputs. Hidden layers are fully-connected with their neighbor layers.

Loss simply calculates the loss between the prediction $\hat{y}$ and real output value y and the backpropagation algorithm is called **Backpropagation Through Time (BTT)**.



The above image, shows the different possible types of RNN sequences, where the bottom is the network input and the top is the network output.

one to many is used for image captioning, i.e. generates a sequence of words that represents the input image.

many to one is mainly used for text classification (e.g. spam/not spam) or sentence correctness checking (e.g. verb time).

many to many is commonly used for language translation (the input layers as a whole can be seen as a decoder, while the output layers as a single decoder), but also for automatic named entity extraction, depending on the configuration.

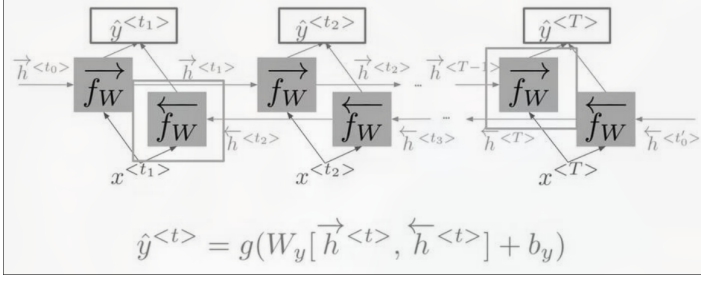


Figure 5.1: Bidirectional RNN example

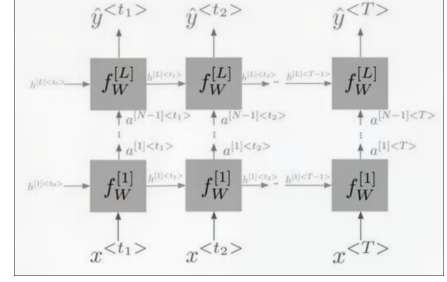


Figure 5.2: Deep RNN example

The equations of RNNs are (make reference to the first RNN image at the beginning of *chapter 5*):

$$a^{<t>} = g_t(W_{aa}a^{<t-1>} + W_{ax}x^{<t>} + b_a) = g_t(W_a[a^{<t-1>}, x^{<t>}] + b_a)$$

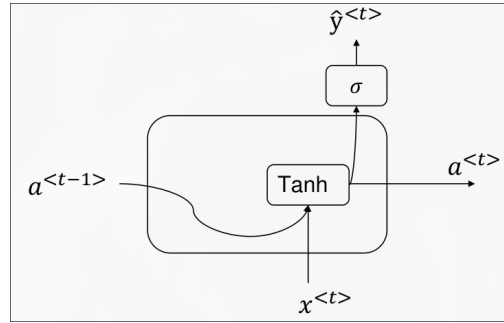
$$\hat{y}^{<t>} = g'_t(W_{ya}a^{<t>} + b_y)$$

where usually g_t is a ReLU or a Tanh function and g'_t is a sigmoid function.

Furthermore, we have that $W_a = [W_{aa}W_{ax}]$ and $[a^{<t-1>}, x^{<t>}] = \begin{bmatrix} a^{<t-1>} \\ x^{<t>} \end{bmatrix}$, in fact:

$$[W_{aa}W_{ax}] \begin{bmatrix} a^{<t-1>} \\ x^{<t>} \end{bmatrix} = W_{aa}a^{<t-1>} + W_{ax}x^{<t>}$$

We introduced a new syntax for the formulas, because it represents better a state in a RNN in a more compact view, like in the image below:



5.1 GRU - Gated Recurrent Unit

An issue with the approach used also in the example at the beginning of *chapter 5*, is that we have different hidden states, one for each input, so researchers tried to find a more compact solution, that didn't require every time a new state for every input. **Gated Recurrent Unit** is born as a solution for this problem, which is a particular hidden state because has an additional variable $c^{<t>}$ (equal to $a^{<t>}$ but present only in this unit) called **memory cell** and its job is to remember some important information relative to the time t .

The **mechanism** of this unit for a specific time (step) t in the network is:

1. find a **candidate of c** , which represents the hidden state t , and which formula is

$$\tilde{c}^{(t)} = \tanh(W_c [c^{(t-1)}, x^{(t)}] + b_c)$$

2. define a **gate u** s.t.

$$\Gamma_u = \sigma(W_u [c^{(t-1)}, x^{(t)}] + b_u)$$

where $\Gamma_u \in \{0, 1\}$ and u is referred as “update”.

3. lastly, **update** the value of c using the formula

$$c^{(t)} = \Gamma_u \cdot \tilde{c}^{(t)} + (1 - \Gamma_u) \cdot c^{(t-1)}$$

$\Gamma_u=1$	$\Gamma_u=0$	$\Gamma_u=0$	$\Gamma_u=0$	$\Gamma_u=0$		$\Gamma_u=1$	$\Gamma_u=0$
$c^0=1$	$c^1=1$	$c^2=1$	$c^3=1$	$c^4=1$		$c^{N-1}=0.8$	$c^N=0.8$
The	cat	which	already	ate		was	full

As we can see in the example above, the value of the memory cell c remain the same when the value of the gate u is 0 and that's because in the updating formula

$$\begin{aligned} c^{(t)} &= \Gamma_u \cdot \tilde{c}^{(t)} + (1 - \Gamma_u) \cdot c^{(t-1)} = \\ &= 0 \cdot \tilde{c}^{(t)} + (1 - 0) \cdot c^{(t-1)} = \\ &= 0 + 1 \cdot c^{(t-1)} \\ &= c^{(t-1)} \end{aligned}$$

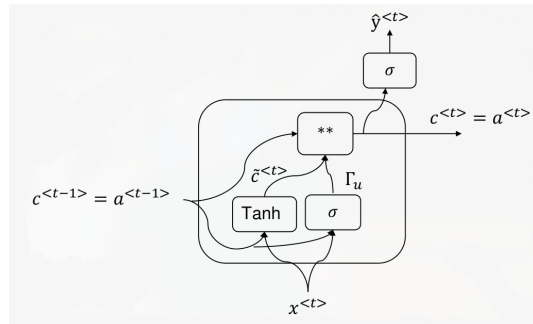
the value of $c^{(t)}$ is equal to the previous value $c^{(t-1)}$ if $\Gamma_u = 0$, but when instead is 1, the value of $c^{(t)}$ is calculated like in a standard hidden state, because

$$\begin{aligned} c^{(t)} &= \Gamma_u \cdot \tilde{c}^{(t)} + (1 - \Gamma_u) \cdot c^{(t-1)} = \\ &= 1 \cdot \tilde{c}^{(t)} + (1 - 1) \cdot c^{(t-1)} = \\ &= \tilde{c}^{(t)} + 0 \cdot c^{(t-1)} \\ &= \tilde{c}^{(t)} \end{aligned}$$

and $\tilde{c}^{(t)}$ formula is the same of $a^{(t)}$ of a standard hidden state. For this reason, like we can see in the picture, the value of $c^{(t)}$ changes and the network knows that the previous cell was empty.

$c^{(t)}$ behaves like a weighted combination of the previous states and the candidate state ($\tilde{c}^{(t)}$).

GRU diagram:



The **Full GRU**, is a variant of the typical Gated Recurrent Unit just seen, in which is used an additional gate r (stands for “relevance”), which formula is

$$\Gamma_r = \sigma \left(W_r \left[c^{\langle t-1 \rangle}, x^{\langle t \rangle} \right] + b_r \right)$$

and has been added by researchers, as a way to solve the vanishing gradient problem.

5.2 LSTM - Long Short-Term Memory

The formulas are:

$$\tilde{c}^{\langle t \rangle} = \tanh \left(W_c \left[a^{\langle t-1 \rangle}, x^{\langle t \rangle} \right] + b_c \right)$$

$$\Gamma_u = \sigma \left(W_u \left[a^{\langle t-1 \rangle}, x^{\langle t \rangle} \right] + b_u \right)$$

$$\Gamma_f = \sigma \left(W_f \left[a^{\langle t-1 \rangle}, x^{\langle t \rangle} \right] + b_f \right)$$

$$\Gamma_o = \sigma \left(W_o \left[a^{\langle t-1 \rangle}, x^{\langle t \rangle} \right] + b_o \right)$$

$$c^{\langle t \rangle} = \Gamma_u \cdot \tilde{c}^{\langle t \rangle} + \Gamma_f \cdot c^{\langle t-1 \rangle}$$

$$a^{\langle t \rangle} = \Gamma_o \cdot c^{\langle t \rangle}$$

where Γ_u is the gate **update**, Γ_f is the gate **forget** and Γ_o is the gate **output**.

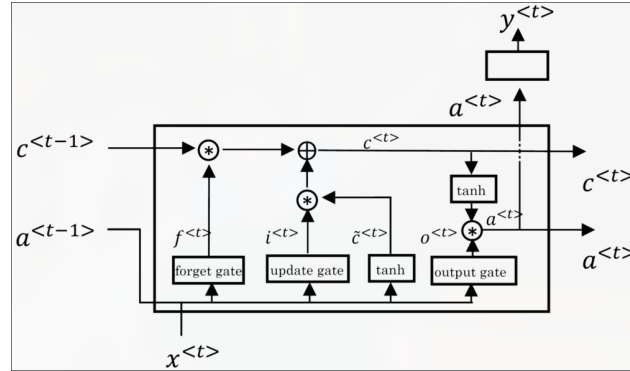


Figure 5.3: LSTM schema

If we train a LSTM with a vocabulary of 10000 words and 100-dimensional activations, the dimension of Γ_u at each time step is 100.

Chapter 6

NLP - Natural Language Processing

Possible applications areas are:

- **Question answering:** receive a question, find the document more suitable for the question, extract the data and formulate an answer.
- **social media intelligence:** like sentiment analysis, so being able to identify what are the general emotions of the people about an action or a topic.
- **machine translation:** translate in real-time virtual communications (e.g. calls).
- **auto-responder:** reply to common messages or e-mails, identified through the matching over a template.
- **fake-news detection:** detect false information, by scanning text and confronting it with others, and also by checking the publisher and the author.

6.1 Word Representation

Word representation techniques represent words as feature vectors. Combined with Deep Learning these techniques are state of the art in NLP in almost all the tasks.

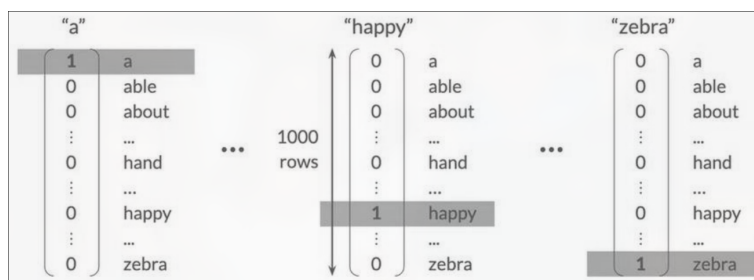
The simplest way to represent words as numbers is for a given vocabulary to assign a unique integer to each word, like in the image on the side.

The set of unique words used in the text corpus is referred to as the **vocabulary**.

This approach is simple, but has the **downside** of having an ordering with **little semantic sense**.

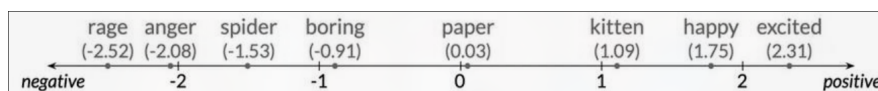
Word	Number
a	1
able	2
about	3
...	...
hand	615
...	...
happy	621
...	...
zebra	1000

Another approach is called **one-hot vectors**, where each word in the vocabulary is mapped in a vector. It's initialized with all 0's and to select a word we just set to 1 the value of the corresponding word, like in figure.

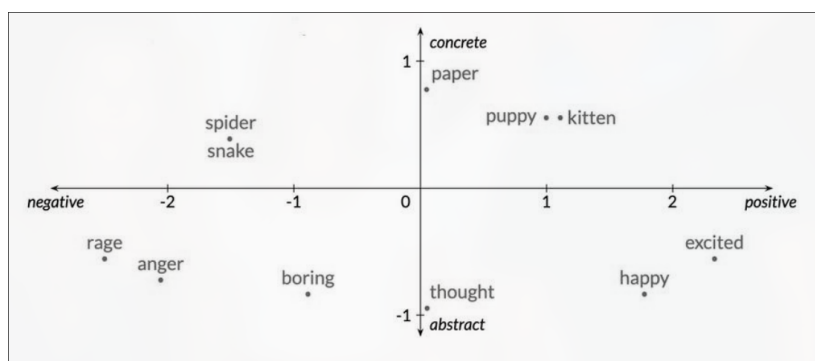


With this approach the ordering isn't needed, but arises new problems, like the **huge vector size** for big vocabularies and the **absence of an embedded meaning** in the structure.

Word embeddings took the approach of one-hot vectors, but fixed the embedded meaning problems, in fact are vectors which also carry the meaning. Words are mapped in the vector, like they're placed over the x-axis, because more a word is on the left (at the beginning of the vector), more has a “negative” meaning, while on the opposite, on the farthest right are placed the “positive” words. For this reason “negative” words have a negative value, while “positive” words have a positive one and the closer is a word to the middle (the 0 of the x-axis), more neutral is their meaning.



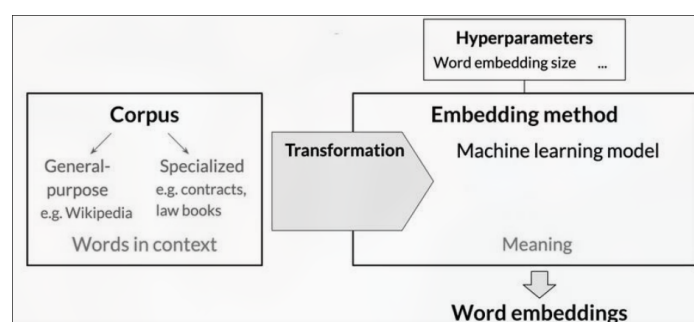
Researchers extended this approach by adding also a y-axis, associated with the level of concreteness (positive for “concrete”, negative for “abstract”), like in the figure:



We can also increase the dimensionality over 2, adding more layers, but this will increase the difficulty of choosing a value for each word.

This final approach has an embedded meaning, because it orders word while creating a semantic distance and all while keeping a low dimensionality.

If we wanted to capture the full range of variation and meaning over our vocabulary of size n , also the word embeddings vector should have a size of n (e.g. 1000 words, 1000 embeddings).



The figure above is the schema of the word embedding process:

1. inputs are the text corpus and some hyperparameters, like the word embedding size (usually 300/500).
2. through a machine learning model, words are arranged with the **embedding method**.
3. lastly, are outputted all the word embeddings, which is a set of numbers (points in our feature space).

6.2 Word2Vec

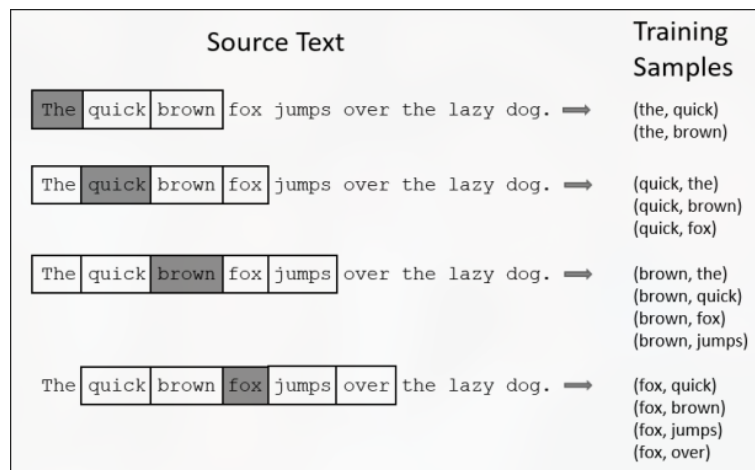
Word2Vec network is a technique for building a rich semantic word embedding space and implements the idea that two words have similar word embedding representations if they have similar contexts.

This network is born as a solution for the following problem:

given a vocabulary and an input word, what are the probabilities of each vocabulary word that, in a possible sentence, is “near” to the input word?

where “near” is represented using a sliding window and its size corresponds to the number of words that precede or follow the input word. Usually the size is 5 and this means that the network will look for 5 words behind and 5 words ahead of the input word.

To train the network and create a dataset, the researchers made the opposite: from a sentence they used the sliding windows, to generate all the tuples of the type $(input_word, near_word)$, like in the figure below (example with window size 2):



The network trained on this sentence, if tested with the word “quick” should return high probabilities for the words “the”, “brown” and “fox” and values as low as possible for the other words. The network is going to learn the statistics from the number of times each pairing shows up.

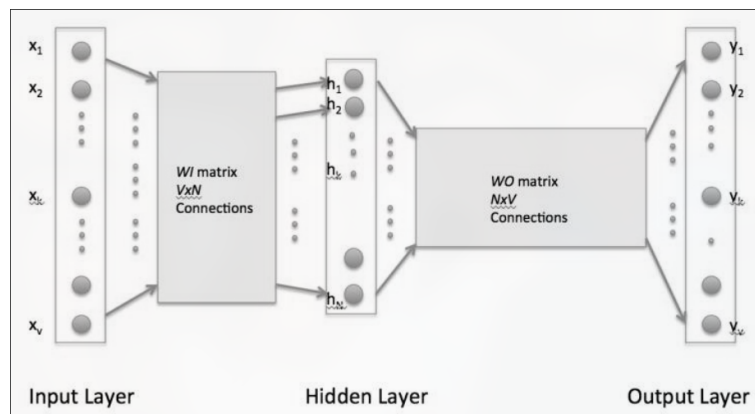


Figure 6.1: Word2Vec network model

The network, schematized in the Figure 6.1, is composed only of an input layer, an intermediate hidden layer and an output layer. Both **input** and **output** have the **same size**, because in output we have the vocabulary, where each word is paired with a probability, and in input we have a one-hot vector with the words, so has the same size of the vocabulary and is set to 1 only in the position of the input word. In the hidden layer, the number of features usually is around 300, so the weight matrix that represents the hidden layer will be $vocabulary_size \times 300$. This network particularity is that has **no activation function** on the hidden layer neurons, while in the output layer is applied a **softmax function**, in order to obtain probabilities in the range $[0,1]$.

In real vocabularies, frequent words like conjunctions, pronouns, ecc (e.g. “the”) are not present and also all those words that always come together (e.g. “New” “York”), are treated as a single word.

6.3 Transformers

BERT (Bidirectional Encoder Representations from Transformers) is a paper published by researchers at Google AI Language. BERT’s key technical innovation is applying the **bidirectional training** of Transformer, a popular attention model, **to language modelling**. In short BERT consists of only the **Encoder** part of the Transformer introduced in the paper “Attention Is All You Need”, with some novel modifications.

Transformer was developed for solving text translation problem and its architecture consists of an **encoder** and a **decoder**, where the encoder’s output is given as input to the decoder.

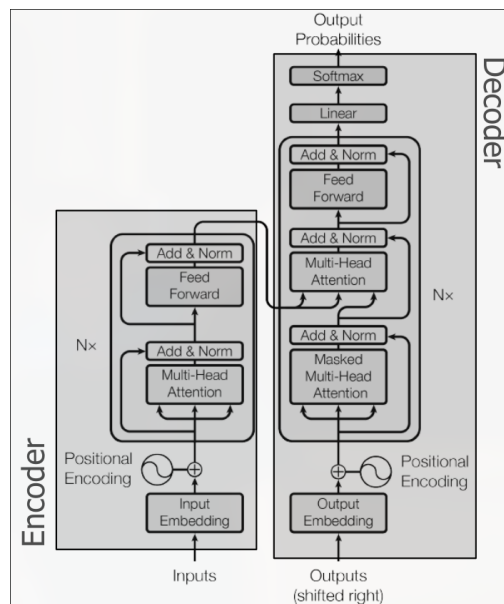
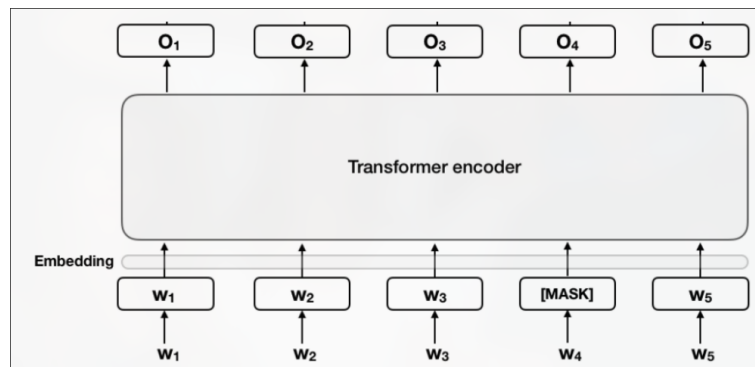


Figure 6.2: Transformer “Attention is all you need”

6.3.1 Encoder

The input is a sequence of tokens, which are first embedded into vectors and then processed in the neural network. The output is a sequence of vectors of size H , in which each vector corresponds to an input token with the same index.



Because the size doesn't change, we can stack multiple encoders creating the so-called **information flow**. The architecture is the following:

1. The model represents each token as a vector of *embedded_dimension* size. We have a matrix of dimension $input_length \times embedded_dimension$ for a specific input sequence (sentence). The *input_length* corresponds to the number of words.
2. Is added positional information to each token, called **positional encoding**, which, at the beginning of the training, are initialized with random numbers. The matrix size remains the same.
3. The data goes through N encoder blocks. After this, we obtain the same matrix dimension.

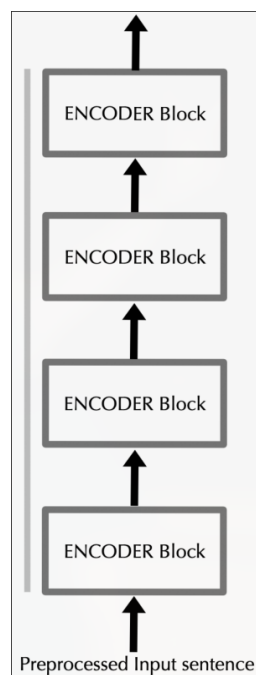


Figure 6.3: example with $N = 4$

Points 1 and 2 are done during the **input preprocessing**. The blocks do not share weights.

The transformation from words to vectors is done like in RNNs and the steps are:

1. **tokenization**: split the words and also punctuation (*e.g.* “?” is important for the phase structure and meaning) in the sentence.
2. **numericalization**: process of mapping each token to a unique integer in the vocabulary.
3. **word embeddings**: each word is mapped to a vector of size *embedded_dimension* that the model will learn during the training and the elements of those vectors are treated as model parameters and are optimized with back-propagation just like any other weights.

The **padding** is added to the input sequences in order to keep the **same size** for each batch.

Each **block** in the encoder is in charge of **finding relationships** between the input representations and encode them in its output. This **iterative process** through the blocks guarantees that the neural network **captures complex relationships** between words.

6.3.2 Multi-Head Attention

The transformer uses **multi-head attention** and this means that it computes the attention **multiple times with different weights** and then concatenates the results together.

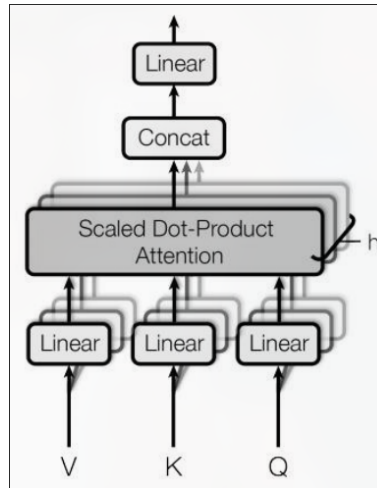


Figure 6.4: multi-head attention schema

In the figure above is represented the multi-head attention mechanism, which takes in input multiple matrices of the same size (*e.g.* 516×300) that are processed in parallel through linear functions (attention functions), which result is merged into one single output called **head** and used in the **scaled dot-product attention**. That’s the origin of this process’ name, in fact we have the concatenation of multiple heads (in the image there is a total of h heads) produced by the attention functions.

The formula is:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) \cdot W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$.

Let's rename the following products as $QW_i^Q = Q_i$, $KW_i^K = K_i$ and $VW_i^V = V_i$, then the Attention's formulas is:

$$\text{Attention}(Q_i, K_i, V_i) = \text{Softmax} \left(\frac{Q_i \cdot K_i^T}{\sqrt{d_k}} \right) \cdot V_i$$

Remember Q_i and K_i are different projections of the tokens into another dimensional space (d_k). Therefore, we can think about the dot product of those projections as a **measure of similarity** between tokens projections.

If we call v_i and u_j the projections of the i-th token and the j-th token through Q_i and K_i respectively, their dot product can be seen as:

$$v_i u_j = \cos(v_i, u_j) \|v_i\|_2 \|u_j\|_2$$

where $\|v_i\|_2$ is the normalization of the vector v_i .

This product measures how similar are the **directions** of v_i and u_j , and how large are their **lengths** (the closest the direction and the larger the length, the greater the dot product).

The network will learn through the training process which **relationships** are more useful and will relate tokens to each other, based on these relationships. We can think about the resulting representation of word as a weighted combination (centroid) of the projected vectors through V_i of the input tokens.

The **position-wise feed-forward** component (that we can see in Figure 6.2) just takes a fully-connected linear layer and applies a ReLU to it.

The **add & norm** component applies a dropout at 10% in the input and the result is added to the original input. It's used to keep the information already learned while also learning new relationships with the remaining tokens.

6.3.3 Training

When training language models, there is a challenge of defining a **prediction goal**. Many models predict the next word in a sequence, but it's a directional approach which inherently **limits context learning**. To overcome this challenge, BERT uses two training strategies simultaneously: **Masked Language Model** and **Next Sentence Prediction**.

Masked Language Model Given a sequence of words, one word is masked and then the sequence is given in input to the model. Then, if the model predicts correctly the word ID, the weights remain the same, otherwise, the weights are updated, *i.e.* the embedding value of the masked word in the dictionary is replaced with the embedding output of the model for the masked word. Before feeding word sequences into BERT, 15% of the words in each sequence are replaced with a `[MASK]` token.

Next Sentence Prediction Find if two or more sentences that compose the sequence of words makes sense or not, *i.e.* a sentence is the following of the previous one. There is no human interaction or preparation, we can just automate all the process by randomly selecting the masked word and the sentences sequence, from pieces of given piece of text.

6.3.4 Fine-Tuning

At the end of the training we obtain a pre-trained BERT model, that can be used for a wide variety of language tasks, by only adding some small layers to the core model and this is called **fine-tuning**. This process differs based on the final task we need for the model:

- **Classification** - given in input a sentence to a model (encoder or decoder), it outputs a classifier; a set with the probabilities of belonging to a specific class in the set.
- **Question Answering** - given in input a question and a text related to the question, the model through a regression should be able to compose an answer using the information extracted from the text (*i.e.* returns the position of the answer in the text)
- **Named Entity Extraction** - given a text, the model should be able to classify each word in a specific class (entity).
- **Named Entity Recognition** - is similar to the previous, with the difference that instead of finding the entities for the text, it recognizes the words that belong to specific entities.

6.3.5 Decoder

Has similar components to the encoder, except for a **Masked Multi-Head Attention** component.

6.4 GPT

Instead of using the encoder, for the GPT they used the **decoder**. GPT differentiate from BERT because, given an input text, it tries to answer by finding each time the **most correct word that follows** the preceding ones in the sentence. That's the reason why using a GPT we have an answer composing animation.

6.4.1 Masked Self-Attention

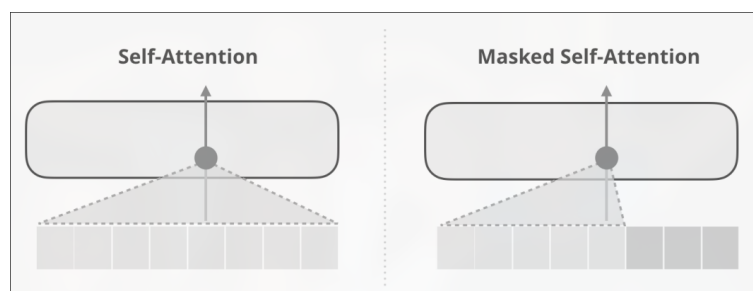


Figure 6.5: Plain vs. Masked self-attention

Every time we want to predict a new word based on what we saw before. This is crucial because in a GPT model we can't use future information (*i.e.* the following words). This component implements a mask that shows only words until a certain point and the following are hidden, like in Figure 6.5.

6.4.2 ChatGPT

Has been developed following this growth steps:

1. Language Model (Self-Supervised Learning)
2. Question Answering (Supervised Learning)
3. Reinforcement Learning from Human Feedback (RLHF)

As a final development step, the researchers engineered a prompt in a way that the model could interpret correctly the input:

- writing clear and specific instructions, using delimiters (like ‘, “”, <>, etc.).

Chapter 7

GNN - Graph Neural Networks

Are used for representing:

- **molecules**
- **images**: graphs with regular structure (e.g. squared) where each node represents a pixel of the image and is connected with the adjacent nodes. Each single node is a 3-dimensional vector representing the RGB value.
- **text**: by indexing each character or word into a node, we can represent sentences as a sequence of nodes through a Direct Acyclic Graph, where words are connected with edges.
- **social networks**: each people/group of people is represented as a node and the relationships are the edges.

7.1 Predictions on graphs

- **graph-level** task: predict the property of an entire graph, e.g. detect the nodes with 2 rings (Figure 7.1)
- **node-level** task: concerned with predicting the identity or role of each node within a graph (e.g. Zach's karate club: club schism, find which members (nodes) followed the first instructor and those that followed the other, *i.e.* identify the split of the nodes in the graph).
- **edge-level** task: used for example in image inferring for film scene understanding, *i.e.* detect objects or people and identify their current action.

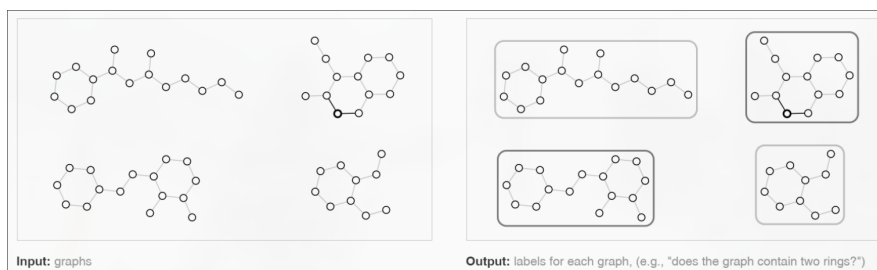


Figure 7.1: Example of graph-level task

7.2 GNN Models

GNN's adopt a “graph-in, graph-out” architecture, meaning that these model types accept a graph as input and progressively transform these embeddings, without changing the connectivity of the input graph. As is common with neural networks modules or layers, we can stack these GNN layers together.

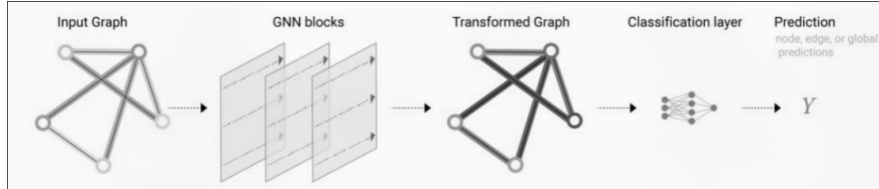


Figure 7.2: End-to-end prediction task with a GNN model

In the block the GNN uses a separate multilayer perceptron (MLP) on **each component** of the graph. Thus for each node and edge the **MLP** is applied and we obtain the learned **embeddings** for each of those, that in the overall make up the transformed graph.

Now, we can make predictions **over the transformed graph** that has been outputted and the approach is straightforward: for each node embedding, we **apply a linear classifier**.

However, it is not always so simple because, for example, we might have information about the graph only stored in the edges and none in the nodes, but we still need to make predictions on nodes. A solution is to **pool** information from the edges and give it to the nodes and this happens in two steps:

1. For each item to be pooled, we **gather** each of their embeddings (vector) **and concatenate** them (matrix).
2. The gathered embeddings are **then aggregated**, usually via a sum or mean operation, *i.e.* each line of the matrix becomes a single cell, thus the matrix becomes a vertical vector.

The pooling operation is represented with the symbol ρ (rho) and the pooling from edges to nodes $\rho_{E_n \rightarrow V_n}$.

7.3 Message Passing

Until now, we were **not using the connectivity** of the graph, instead we were using the information of each component independently. Instead, we could make **more sophisticated predictions** by using pooling within the GNN layer, in order to make our learned embeddings aware of graph connectivity. We can achieve this by using **message passing**, where neighbor nodes or edges exchange information and influence each other on their updated embeddings. This technique is split in three steps:

1. gather all the neighbors' node embeddings (or messages)
2. aggregate all messages (e.g. sum)
3. pass all through an update function, usually a learned neural network

Message passing can occur between either nodes or edges indifferently.

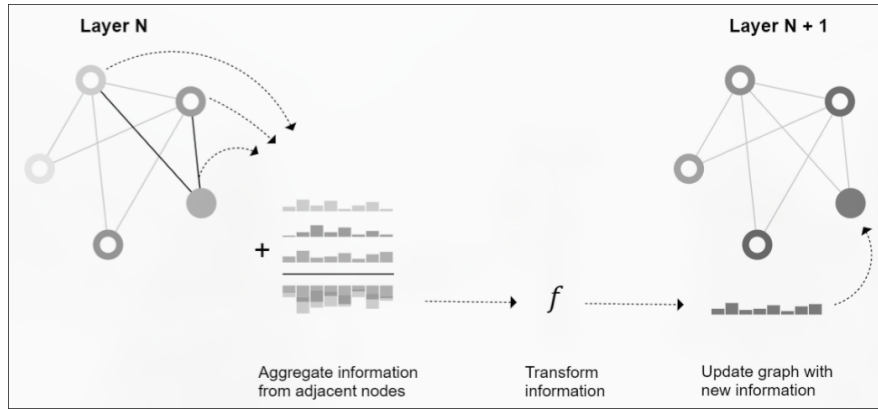


Figure 7.3: Example of Message Passing over the full node in the image

The difference with the previous and easier approach is that to the transformation function are given in input multiple information (the one of the node plus those of the adjacent nodes), while before we only gave the nodes' one.

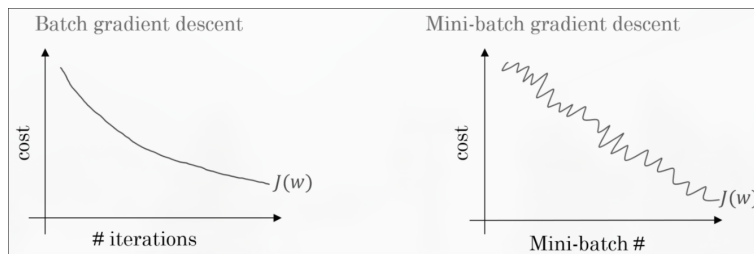
In essence, Message Passing is similar to Convolution because are operations that aggregates and process information of an element's neighbors in order to update the element's value. In graphs, the element is a node, while in images, the element is a pixel and this makes some difference because in images each pixel has a fixed number of neighbors, while in graphs this number is variable. However, with graphs, we're able to pass multiple GNN layers together, so a node can incorporate information from the entire graph, e.g. after three layers, a node has information about the nodes three steps away from it.

Chapter 8

Optimization

8.1 Batch vs. Mini-Batch Gradient Descent

When you are using the Gradient Descent is used the entire training set to compute the Cost function and the derivatives. However, it's prohibitive when you are dealing with very large-scale datasets. For this reason we can split the dataset into smaller mini-batches which the Gradient will compute one at a time separately. This strategy is obviously called **Mini-Batch Gradient**.



As we can see in the figure above, the mini-batch gradient curve isn't smooth as the single-batch because we can have mini-batches that are more difficult than others.

To the opposite side of the Batch Gradient Descent approach there is the **Stochastic Gradient Descent**, which works over batches of size 1, so obviously doesn't use vectorization because takes one element at a time, but it's really slow. Normal Batch Gradient Descent takes the maximum size m of the input dataset and uses it all, thus requiring a huge memory space for vectorization. Mini-Batch Gradient Descent is the intermediate solution between these two extremes.

If we have a small training set, like 2000 samples, we can just use Batch Gradient Descent, while, if we have a larger training set, the typical mini-batch sizes are 64, 128, 256, 512 and 1024.

8.2 Algorithms

8.2.1 Exponentially weighted average

$$V_t = \beta V_{t-1} + (1 - \beta)\theta_t$$

where V_t is the weighted average value at time t , β is the weight factor and θ is the recorded value at time t .

Because this formula doesn't return good result with low value for t , we can use **bias correction** to adjust the value and the formula is:

$$\tilde{V}_t = \frac{V_t}{1 - \beta^t}$$

where $\tilde{V}_t$ is the corrected V_t value.

8.2.2 Gradient Descent with Momentum

Almost always works empirically faster than standard gradient descent algorithm. For each iteration compute $\frac{d\mathcal{L}}{dW}$ and $\frac{d\mathcal{L}}{db}$ and the new values of dW and db are

$$V_{dW} = \beta V_{dW} + (1 - \beta) \frac{d\mathcal{L}}{dW}$$

$$V_{db} = \beta V_{db} + (1 - \beta) \frac{d\mathcal{L}}{db}$$

where a good value for β is 0.9 and the final updated values for W and b are:

$$W = W - \alpha V_{dW}$$

$$b = b - \alpha V_{db}$$

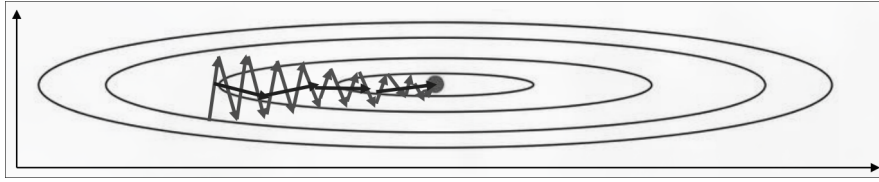


Figure 8.1: The Y-axis is b and the X-axis is W

In the above figure there is the graphic comparison between the plain gradient descent (more up-down arrows) against the version with momentum (more linear arrows).

8.2.3 RMSprop

Also this approach updates $\frac{d\mathcal{L}}{dW}$ and $\frac{d\mathcal{L}}{db}$ for each interaction, but the formulas are slightly different:

$$S_{dW} = \beta S_{dW} + (1 - \beta) \left(\frac{d\mathcal{L}}{dW} \right)^2$$

$$S_{db} = \beta S_{db} + (1 - \beta) \left(\frac{d\mathcal{L}}{db} \right)^2$$

and also in this case a good value for β is 0.9 and the final updated values for W and b are:

$$W = W - \alpha \cdot \frac{\frac{d\mathcal{L}}{dW}}{\sqrt{S_{dW} + \varepsilon}}$$

$$b = b - \alpha \cdot \frac{\frac{d\mathcal{L}}{db}}{\sqrt{S_{db} + \varepsilon}}$$

and a good value for ε is 10^{-8} and is added to avoid the division by 0.

By squaring $\frac{d\mathcal{L}}{dW}$ and $\frac{d\mathcal{L}}{db}$, we make the variation of those value more relevant in the result.

8.2.4 ADAM - Adaptive Moment Estimation Optimizer

ADAM combines the algorithms that we have just seen until now. The formulas are:

$$\begin{aligned}V_{dW} &= \beta_1 V_{dW} + (1 - \beta_1) \frac{d\mathcal{L}}{dW} \\S_{dW} &= \beta_2 S_{dW} + (1 - \beta_2) \left(\frac{d\mathcal{L}}{dW} \right)^2 \\ \tilde{V}_{dW} &= \frac{V_{dW}}{1 - \beta_1^t} \\ \tilde{S}_{dW} &= \frac{S_{dW}}{1 - \beta_2^t} \\ W &= W - \alpha \cdot \frac{\tilde{V}_{dW}}{\sqrt{\tilde{S}_{dW} + \varepsilon}}\end{aligned}$$

like we can see the V_{dW} is taken from the Gradient Descent with Momentum, the S_{dW} is taken from the RMSprop and then are calculated the bias corrected values both for V_{dW} and S_{dW} that are used to update the value, with the RMSprop formula. α needs to be tuned. The formulas are equal also for b , only by substituting dW with db and are not written for redundancy.

8.3 Learning Rate

Tuning the learning rate (α) is crucial because with a low value we move too slow for reaching the local minimum, while with high values we end jumping around almost randomly without making real and consistent improvements in reaching the local minimum. We can observe the graphical differences while finding the local minimum in the figure below:



To tune the learning rate, right now is in vogue the **learning rate decay**, which has formula

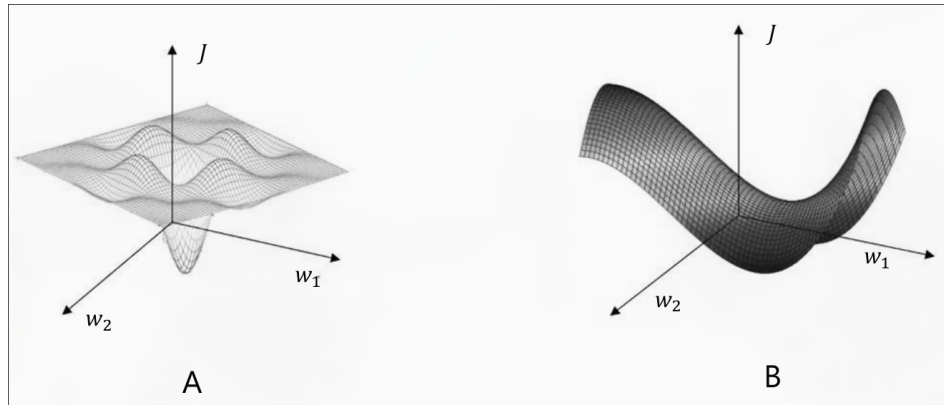
$$\alpha = \frac{1}{1 + \text{decay_rate} \cdot \text{current_epoch}} \cdot \alpha_0$$

Instead a **not good** learning rate formula is:

$$\alpha = e^t \cdot \alpha_0$$

8.4 Local Optima in Neural Networks

In a high-dimensional space often, instead of having a shape with a local point like in figure A, we have space that is more like B, which is not so convenient to manage because it's more difficult to find a real local minimum like in A.



In a scenario like figure B the algorithms that we've seen before work better than the plain Gradient Descent.

Chapter 9

Loss functions

We’ve just seen in the chapters before the **Binary Cross-Entropy Loss**, used for Binary Classification and the **Mean Squared Error Loss**, used in Regression.

9.1 Softmax Activation Layer

In multi-class classification, the last layer of a neural network is often a Softmax layer, and it means that the network will output a probability distribution over the classes. Its formula is:

$$a^{[i]} = \frac{e^{z^{[i]}}}{\sum_{j=1}^n e^{z_j^{[i]}}}$$

where n is the number of classes and $z^{[i]} = W^{[i]} \cdot a^{[i-1]} + b^{[i]}$.

Again, this loss function is used for **multi-class classification**.

9.2 Cross-Entropy Loss

Is the non-binary version, commonly used to quantify the difference between two probability distributions in multi-class classification, and its formula is:

$$L(W, b) = - \sum_{i=1}^m y^{(i)} \cdot \log(h_{W,b}(x^{(i)}))$$

where m is the number of training samples, $y^{(i)}$ is the ground truth output distribution and $h_{W,b}(x^{(i)})$ is the predicted output distribution.

The Cross-Entropy Loss focuses on the **positive class**, because the negative classes are potentially infinite.

9.3 Ranking Loss Function

Unlike other loss functions, such as Cross-Entropy Loss or Mean Square Error Loss, whose objective is to learn to predict directly a label, a value or a set of values given in input, the objective of Ranking Loss is to predict **relative distances** between inputs. This task is often called **metric learning**.

This function computes a **similarity score** between data points to be able to classify the data, using for example a binary classification, like “similar” and “dissimilar”.

9.4 Pairwise Loss Function

Is a ranking loss function in which we set an **anchor** sample, for be able to train our model by comparing the anchor against positive (similar) and negative (dissimilar) samples. For example, a face detection for a specific person, we use a photo of him as an anchor and other images, maybe with different angles and light condition to train the model to predict that is the same person, while also using negative samples, thus samples of other people that the model should identify as different faces.

In this way, we don't work only on strengthening our model over similarities, but also over dissimilarities.